GadVata - Maharashtra's Comprehensive Trekking & Safety Platform

Submitted in Partial Fulfillment of Requirements
for the Degree of

Bachelor of Computer Applications

By

Vedant Prakash Chavan

Seat No: 31011223010

Guide

Ms. Anushree Sukhi

Somaiya School of Basic and Applied Sciences
Somaiya Vidyavihar University
Vidyavihar, Mumbai - 400 077

2023-26

Somaiya Vidyavihar University

Somaiya School of Basic and Applied Sciences

Certificate

This is to certify that the project report on dissertation entitled **GadVata** is Bonafide record of the dissertation work done by **Vedant Chavan** in the year 2025-26 under the guidance of Ms. **Anushree Sukhi**, Department of Information Technology and Computer Science in partial fulfillment of requirement for the Bachelor of Computer Applications degree in Somaiya School of Basic and Applied Sciences of Somaiya Vidyavihar University.

Guide / Co-Guide	Programme Coordinator
Head of Department	Director

Date:

Place: Mumbai-77

Somaiya Vidyavihar University

Somaiya School of Basic and Applied Sciences

Certificate of Approval of Examiners

This is to certify that the project report on dissertation entitled **GadVata** is Bonafide record of the dissertation work done by **Vedant Chavan** in partial fulfillment of requirement for the Bachelor of Computer Applications degree of Somaiya Vidyavihar University.

_____ _____

External Examiner /Expert Internal Examiner/Guide

Date:

Place: Mumbai-77

Somaiya Vidyavihar University

Somaiya School of Basic and Applied Sciences

DECLARATION

I declare that this written report submission represents the work done based on my and / or others' ideas with adequately cited and referenced the original source. I also declare that I have adhered to all principles of academic honesty and integrity as I have not misinterpreted or fabricated or falsified any idea/data/fact/source/original work/ matter in my submission.

I understand that any violation of the above will be cause for disciplinary action by the college and may evoke the penal action from the sources which have not been properly cited or from whom proper permission is not sought.

Signature of the Student

Vedant Prakash Chavan

Name of the Student

31011223010

Seat No.

Date:

Place: Mumbai-77

EXECUTIVE SUMMARY

Maharashtra is home to some of the most historically significant and geographically challenging forts in India. However, the ecosystem surrounding fort exploration is highly fragmented. Trekkers struggle with scattered route information and safety concerns, while local vendors, guides, and restaurants at the base villages lack a unified digital platform to offer their services. Furthermore, emergency response during trekking accidents is often delayed due to the lack of real-time GPS tracking and medical intelligence.

GadVata Evolution is proposed and developed as a comprehensive, multi-tenant Software-as-a-Service (SaaS) platform to solve these critical infrastructural gaps. The project bridges the gap between historical exploration and modern digital fleet management by creating a unified ecosystem for six distinct user roles: Trekkers, Local Vendors, Restaurants, Trek Guides, Trek Agencies (Organizers), and Forest Rescue Commands.

Extensive literature survey and market analysis show that existing trekking applications primarily focus on static GPS mapping or community forums. GadVata differentiates itself by introducing a dynamic, real-time micro-economy. Through this application, trekkers can view real-time weather APIs, download offline tactical step-by-step route PDFs, and book expedition seats dynamically. Simultaneously, local businesses operate dedicated partner dashboards to manage inventory, update live seating matrices, and process dynamic pricing for guide services based on group sizes.

A major technical milestone of this project is the integration of the Emergency SOS Engine. When triggered, it broadcasts the trekker's exact geolocation coordinates and pre-filled medical history directly to a flashing, haptic-enabled Rescue Command terminal, dramatically reducing emergency response times. The entire ecosystem is monitored by a Root Admin Dashboard that aggregates real-time data from all user nodes, calculates total platform revenue, and features a global lockdown override.

In this report, an exhaustive study on the system architecture, component design, database state management, and real-time interface rendering is presented. The implementation successfully demonstrates how localized unorganized sectors can be digitally transformed into a structured, safe, and highly efficient tourism economy.

Keywords : SaaS Platform, Micro-economy, Geolocation Tracking, Emergency Response System, Multi-Vendor Ecosystem, React Native.

Contents

List of Figures				
List of Tables				
Nomenclature				
1	**Introduction**			
	1.1	Introduction and background of the project		1
	1.2	Objectives of project and Problem Statement		2-3
	1.3	Scope, Advantages, and applicability of project		3-4
	1.4	Organization of the report.		4
2	**Survey of Technologies**			
	2.1	Existing technologies		5-6
	2.2	Technology implemented for the project		6-7
3	**Requirement and Analysis**			
	3.1	Problem Statement with major modules		8
	3.2	Feasibility Study		9-10
	3.3	Software Requirement Specification		
		3.3.1	Functional Requirements	10-11
		3.2.2	Non-functional Requirements	11-12
	3.4	System software and Hardware requirements		12-13
	3.5	SDLC model		14-15
	3.6	Project Scheduling		16-17
4	**System Design**			
	4.1	Use case Diagram		18-24
	4.2	Data Flow Diagram		25-26

	4.4	Activity Diagrams	26-29
	4.5	Project system screens	30-52
5	**Implementation and Testing**		
	5.1	Algorithms and Flowcharts along with major modules of the project	53-55
	5.2	Testing Approaches	56
	5.3	Test Cases and Test Results	57
6	**Results and Discussion**		58-59
7	**Conclusion and Future work**		60-61
References			62
Annexures			

List of Figures

Sr. No.	Name of the Figure	Page No.
1.	Proposed SDLC Model (Agile)	14
2.	Project Gantt Chart / Timeline	17
3.	Use Case Diagram: Trekker Application	19
4.	Use Case Diagram: Partner / Business Portal	20
5.	Use Case Diagram: Rescue & Admin Command	21
6.	Level 0 DFD: Context Diagram	25
7.	Level 1 DFD: Booking & Inventory Engine	25
8.	Entity Relationship (ER) Diagram	26
9.	Activity Diagram: Emergency SOS Flow	27
10.	GadVata Home Screen	30
11.	GadVata Navigation Screen	31
12.	GadVata Trails Screen	32
13.	GadVata Settings Screen	33
14.	Overview Screen	34

15.	Spots Screen	35
16.	Travel Screen	36
17.	Food / Stay Screen	37
18.	Restaurant Screen	38
19.	Guides Screen	39
20.	Vendor Side Interface	40
21.	Restaurant Side Interface	41
22.	Trek Guide Interface	42
23.	Travel Agency Interface	43
24.	Rescue Command Radar Interface	44
25.	Root Admin Master Terminal	45

List of Tables

Sr. No.	Name of the Table	Page No.
3.1	Feasibility Analysis Matrix	9
1.2	Functional Requirements Definition	10
1.3	Hardware & Software Specifications	12-13
1.4	Project Development Schedule	16-17
1.5	Test Case: Authentication & OTP Verification	55
1.6	Test Case: SOS Hardware Trigger	56
1.7	Test Case: Admin Global Lockdown Override	57

Nomenclature

Term / Abbreviation	Meaning
API	Application Programming Interface
DFD	Data Flow Diagram
ER	Entity Relationship
GPS	Global Positioning System
JSON	JavaScript Object Notation
MERN	MongoDB, Express.js, React Native, Node.js
MVP	Minimum Viable Product
OTP	One-Time Password
REST	Representational State Transfer
SaaS	Software as a Service
SDLC	Software Development Life Cycle
SOS	Save Our Souls

This chapter introduces the foundational context and motivation behind the development of GadVata Evolution. It outlines the historical and socio-economic background of Maharashtra's trekking ecosystem, defines the core problem statement, and establishes the primary technical objectives. Additionally, it highlights the scope, advantages, and real-world applicability of the platform, concluding with the structural organization of this dissertation

Chapter 1 : Introduction

1.1 Introduction and background of the project

The Sahyadri mountain range in Maharashtra houses hundreds of historically significant forts that stand as monuments to the Maratha Empire. In recent years, adventure tourism and historical trekking have seen an exponential rise in popularity among the youth and fitness enthusiasts. However, despite this massive influx of foot traffic, the digital and physical infrastructure supporting this ecosystem remains severely outdated and highly fragmented.

Currently, the trekking industry operates entirely in the unorganized sector. Information regarding trek difficulty, route safety, and historical significance is scattered across outdated web forums, unregulated social media pages, and informal messaging groups. More critically, the local economy at the base of these forts—comprising rural food vendors, homestay owners, and local guides—relies heavily on unpredictable, seasonal footfall with zero digital visibility. When trekkers arrive, they face uncertainties regarding food availability, safe drinking water, and authentic historical guidance.

From a safety perspective, mountain trekking inherently carries significant physical risks. In the event of an injury, sudden weather changes, or a trekker losing their way, the current emergency protocol relies on standard cellular calling, which is often unavailable in high-altitude blind spots. Rescue teams receive fragmented, delayed information regarding the victim's exact coordinates and medical history, leading to potentially fatal delays.

To address these critical systemic failures, GadVata Evolution was conceptualized and developed. GadVata is not merely a navigation application; it is a comprehensive, multi-tenant Software-as-a-Service (SaaS) ecosystem. It digitally transforms the unorganized trekking sector by bringing six distinct operational nodes onto a single, synchronized platform: Trekkers, Local Vendors, Restaurants, Trek Guides, Trek Agencies (Organizers), and Forest Rescue Commands. By engineering a platform that handles real-time seat matrices, dynamic pricing, offline tactical routing, and instantaneous SOS telemetry, GadVata establishes a unified, safe, and economically thriving digital infrastructure for Maharashtra’s historical tourism.

1.2 Objectives of project and Problem Statement

Problem Statement

The adventure tourism sector in Maharashtra, specifically regarding the exploration of historical Marathi forts, is currently experiencing massive growth. However, the ecosystem remains entirely unorganized. Local base village agencies and guides rely on fragmented communication (WhatsApp groups) and manual ledger bookkeeping. Simultaneously, trekkers venture into mountainous terrains that inherently lack consistent mobile network coverage, exposing them to severe logistical and physical risks.

The primary problem this project addresses is the lack of a centralized, data-driven ecosystem for fort exploration in Maharashtra. During the initial research phase of this project, it became evident that trekkers face high risks due to unsafe trails, sudden weather changes, and a complete lack of offline tactical route guides. When network connectivity drops, trekkers are often left completely blind, relying on outdated blogs or word-of-mouth directions.

Simultaneously, the local economy at the base villages comprising food vendors, small restaurants, and local guides suffers from a severe lack of digital discovery. Because there is no centralized inventory or booking management system, these businesses operate on unpredictable daily footfall, leading to economic instability and food wastage. Furthermore, the most critical issue identified was emergency rescue operations. In the event of a medical emergency or an accident on the fort, rescue operations are severely handicapped by high-latency communication. Traditional emergency calls fail in zero-network zones, and even when they connect, there is a massive delay in transmitting precise geolocation and critical medical data to the rescue teams.

Objectives of the Project

To systematically solve the aforementioned problems, the GadVata platform was engineered from the ground up with the following core technical objectives:

Unified Multi-Tenant Architecture: To design and deploy a centralized mobile application featuring dedicated, secure, and role-based dashboards for six distinct user types: Trekkers, Vendors, Restaurants, Guides, Trek Agencies, and Rescue units.

Real-Time Micro-Economy & Inventory Engine: To build a dynamic backend system that allows trekkers to order food seamlessly, hire guides based on algorithmically tiered group pricing (e.g., solo vs. group rates), and book expedition seats with live-updating capacity matrices that prevent double-booking.

Tactical Offline Navigation: To develop a system that provisions downloadable, offline-accessible step-by-step PDF route guides, ensuring that critical navigation and historical checkpoints remain accessible even when the device enters a zero-network zone.

Instantaneous SOS Telemetry Engine: To engineer a low-latency Emergency SOS Beacon that completely bypasses traditional call routing. The objective is to instantly inject the user's exact GPS coordinates and pre-registered medical history directly into a localized Forest Rescue Command terminal for immediate dispatch.

Centralized Overseer Control (Admin Terminal): To implement a Root Admin Terminal capable of aggregating live ecosystem data from all active nodes, calculating cross-node revenue, and possessing the ultimate authority to execute a global system lockdown (freezing bookings and marking trails as dangerous) in the event of extreme weather or mountain emergencies

1.3 Scope, Advantages, and Applicability of the Project

The current Minimum Viable Product (MVP) scope of the GadVata Evolution project is strategically restricted to the top 30 most historically significant and highly frequented forts within the Sahyadri range of Maharashtra (including, but not limited to, Rajgad, Torna, Raigad, and Shivneri). This focused scope allows for deep data accuracy and rigorous testing of the multi-tenant architecture.

The system scope encompasses the complete digital lifecycle of a trek. For the trekker, it covers initial fort discovery, real-time API weather checking, pre-booking of travel agency seats, on-fort local vendor transactions, and post-trek community feedback. For the partners, the scope includes live inventory management (e.g., marking items "Out of Stock"), dynamic request approvals, and revenue tracking. The project intentionally excludes out-of-state trekking locations to maintain a highly localized and optimized database structure during this phase of deployment.

Advantages

- **Critical Risk Mitigation:** The direct-to-terminal SOS engine is the most significant advantage of this platform. By transmitting exact coordinates and medical data instantly, it drastically reduces the response time of emergency services, potentially saving lives in high-altitude, high-risk environments.

- **Economic Empowerment for Rural Sectors:** Rural vendors and independent guides are elevated from unorganized laborers to verified digital partners. They are given professional digital storefronts, allowing them to forecast demand, manage their live stock availability, and significantly stabilize their seasonal income.

- **Verified Data Consistency:** Trekkers are no longer forced to rely on fragmented, potentially dangerous advice from random forums. Route safety, physical difficulty levels, and historical context are standardized, verified, and centralized within the GadVata database.

- **Absolute Offline Reliability:** The architecture explicitly accounts for the lack of internet infrastructure in the mountains. The provision of downloadable PDF maps ensures that trekkers have uninterrupted access to critical navigation timelines regardless of cellular network availability.

• **Applicability** The GadVata ecosystem has immediate and widespread real-world applicability. It is directly applicable to adventure tourists, historical researchers, and college trekking groups seeking a safe, organized experience. It serves as a direct digital bridge connecting urban consumers (trekkers) with rural service providers (base village businesses), formalizing a previously shadow economy. Furthermore, the custom Rescue Command module has direct, critical applicability for local police forces, forest rangers, and contracted mountain rescue organizations who require precise, real-time dispatch operations.

1.4 Organisation of the Report

This dissertation report is meticulously organized to logically present the entire development lifecycle of the GadVata ecosystem, from initial concept to final deployment.

Chapter 1 (Introduction): Introduces the foundational context, outlines the historical and socio-economic background of the trekking ecosystem, defines the core problem statement, and establishes the primary technical objectives and scope of the platform.

Chapter 2 (Survey of Technologies): Critically evaluates existing trekking and navigation applications, highlighting their shortcomings. It then extensively details the specific MERN/React Native technology stack, APIs, and libraries chosen to build GadVata.

Chapter 3 (Requirement and Analysis): Details the rigorous feasibility study conducted prior to development. It outlines the exhaustive functional and non-functional requirements, hardware/software specifications, and the Agile SDLC (Software Development Life Cycle) methodology utilized to manage the project sprints.

Chapter 4 (System Design): Presents the complex architectural blueprints of the platform. This includes detailed Use Case diagrams for all six user roles, Data Flow Diagrams (DFDs) tracking the movement of information, Entity Relationship (ER) diagrams mapping the database, and Activity flows mapping complex interactions like the SOS engine.

Chapter 5 (Implementation and Testing): Discusses the core algorithms and the programmatic logic behind major modules (such as the booking engine and dynamic pricing). It also details the rigorous software testing approaches applied, including specific test cases for boundary validation and UI responsiveness.

Chapter 6 (Results and Discussion): Showcases the final graphical user interfaces (Screenshots) of the deployed system and evaluates the operational success, performance, and user experience of the completed features.

Chapter 7 (Conclusion and Future Work): Summarizes the overall impact of the GadVata project and outlines a strategic roadmap for future scalability, proposing advanced features like AI integrations, drone mapping, and broader geographical expansion across India.

Chapter 2. Survey of Technologies

2.1 Existing Technologies

Before engineering the GadVata platform, an extensive literature and market survey was conducted to evaluate the software tools currently used by trekkers and local businesses. The survey revealed that the ecosystem currently relies on a disjointed combination of generic applications, none of which are purpose-built for the unique challenges of the Sahyadri terrain

2.1.1 Mobile Application Development Frameworks

- **React Native**: This is the core open-source framework you used to build the cross-platform mobile application. It allowed you to use a single codebase to render a native UI for both Android and iOS devices.
- **Expo SDK (Version 50+)**: You used Expo as a set of tools and services built around React Native to simplify the development process. It provided the necessary libraries to access native device features and managed the build environment.
- **Expo Router**: This was implemented as the file-based routing library for your app. It allowed you to manage navigation between screens (like the Home Screen and Product Details) using a directory structure similar to web development.
- **React Native Safe Area Context**: You integrated this to ensure that your app's UI elements, like the back button and footer, are correctly positioned away from physical notches and home indicators on modern smartphones.
- **Ionicons (via @expo/vector-icons)**: This library was used to provide the visual iconography across your app, such as the cart and arrow-back icons.

2.1.2 General Purpose Mapping Systems (e.g., Google Maps) While highly effective for urban navigation and paved highways, general-purpose mapping APIs fail significantly in mountain terrains. These systems lack granular topographic data and do not map unpaved, historical trekking trails. Furthermore, they require continuous internet connectivity to render map tiles. In the zero-network zones of forts like Rajgad or Harishchandragad, these applications frequently time out, leaving the user with a blank grid and no directional orientation.

2.1.3 International Trail Applications (e.g., AllTrails, Strava) Applications like AllTrails offer trail mapping, but they are designed primarily for Western demographics. They lack deep, localized historical context regarding Maratha architecture and history. More critically, they are strictly consumer-facing "read-only" applications. They offer absolutely no integration with the local base-village economy. A trekker cannot use these apps to check if a local vendor has drinking water in stock or to book a local guide.

2.1.4 Informal Communication Networks (WhatsApp, Facebook Groups) Currently, the entire commercial aspect of trekking—booking agency seats, hiring guides, and arranging homestays—is conducted via informal social media groups. This presents massive data inconsistency and security risks. Trek agencies manage seat capacities manually on spreadsheets, leading to frequent overbooking. Financial transactions are unregulated, and trekkers have no verified system to check the credentials of a guide or the hygiene standards of a local restaurant.

2.1.5 Standard Emergency Protocols (Cellular 112/100) The most glaring technological failure exists in emergency management. If a trekker is injured, they must physically dial emergency services, explain their location verbally (which is highly inaccurate on a mountain), and wait for dispatch. This high-latency communication protocol results in delayed rescues. There is no existing system that instantly transmits precise hardware GPS coordinates and pre-existing medical profiles directly to a localized forest ranger terminal.

2.2 Technology Implemented for the Project

To overcome the limitations of existing systems, GadVata Evolution was developed utilizing a modern, decoupled client-server architecture. The application leverages the MERN stack integrated with React Native for cross-platform mobile deployment. This specific stack was chosen for its asynchronous processing capabilities, scalable NoSQL database structure, and high-fidelity native mobile rendering.

1. Frontend Application: React Native and Expo React Native was selected as the core frontend framework. Unlike traditional native development (which requires writing separate Java code for Android and Swift code for iOS), React Native allows for a single, unified JavaScript codebase that compiles to native hardware components on both platforms. This drastically reduced the development lifecycle and ensured feature parity across all devices.

- **Expo Framework:** The Expo toolchain was utilized to manage the React Native deployment. Expo provides robust, pre-configured access to native device hardware. For instance, the expo-location module was crucial for capturing high-accuracy GPS coordinates for the SOS engine, while expo-image-picker was utilized to allow trekkers and vendors to upload localized media directly from their device galleries.

- **Dynamic Theme Engine:** A custom state-managed UI layer was engineered to support real-time light/dark mode switching, utilizing dynamic hexadecimal variable injection across all tab navigators and component cards.

2. Backend Architecture: Node.js and Express.js The server-side logic is powered by Node.js, utilizing the Express.js framework. Node.js operates on a single-threaded, non-blocking, event-driven architecture. This is highly critical for GadVata's dynamic booking engine. When hundreds of trekkers attempt to query the seat capacity of a specific trek agency simultaneously, Node.js handles these concurrent API requests efficiently without bottlenecking the server CPU. Express.js was used to construct secure RESTful API endpoints that process authentication, vendor inventory updates, and SOS coordinate routing.

3. Database Management: MongoDB Given the multi-tenant nature of the platform, a rigid relational database (like MySQL) would have caused severe schema limitations. GadVata manages six distinct user types: Trekkers, Vendors, Restaurants, Guides, Agencies, and Admins. Each role requires vastly different data structures. For example, a Restaurant document requires menu_items and stock_status, whereas a Trekker document requires medical_history and emergency_contacts. MongoDB, a NoSQL database, was chosen because it stores data in flexible, JSON-like documents (BSON). This allowed the rapid iteration of user profiles and the efficient querying of nested arrays, which is essential for rendering the live seat matrices and dynamic guide pricing models based on group sizes.

4. External APIs and Libraries To avoid "reinventing the wheel" and to optimize system performance, several industry-standard third-party APIs and libraries were integrated into the platform:

- **Open-Meteo API:** A highly accurate, real-time weather forecasting API. GadVata pings this API asynchronously to display live temperature and precipitation warnings on the fort overview screens, allowing trekkers to make informed safety decisions.

- **PDF Generation & Handling Libraries:** To solve the zero-network issue, the system generates and provisions tactical step-by-step route maps as lightweight PDF buffers. These files are cached locally on the device's storage, enabling full offline rendering of critical navigation timelines.

- **Haptic Feedback APIs:** Integrated specifically for the Rescue Command Terminal and the Admin Dashboard. When a high-priority event occurs (such as an SOS trigger or a Global Lockdown override), the application triggers native device vibration motors to ensure the notification is physically felt by the user, enhancing the urgency of the alert.

Chapter 3 : Requirements & Analysis

3.1 Problem Statement with major modules

As defined in Chapter 1, the core problem is the severe fragmentation of the trekking ecosystem, resulting in safety hazards and economic inefficiencies. To solve this programmatically, the GadVata SaaS architecture was decoupled into four major, interconnected modules. Each module is specifically engineered to handle the unique data workflows of its assigned user role.

1. Trekker Facing Interface (B2C Module) This is the primary consumer application used by trekkers on the mountains.

- **Live Dashboard:** Renders real-time weather APIs, fort difficulty ratings, and historical data.
- **Booking Engine:** Allows users to select dates, input group sizes, and pay for guided treks, updating the global seat matrix.
- **Offline Vault:** Caches downloaded tactical PDF maps into local device storage for zero-network navigation.
- **SOS Emitter:** A hardware-linked panic button that captures precise device GPS coordinates and pre-stored medical profiles.

2. Partner & Vendor Ecosystem (B2B Module) This module acts as the digital storefront and inventory manager for base-village businesses.

- **Live Seat Matrix:** Trek agencies use this to define total capacity. The system automatically locks bookings when the matrix hits zero.
- **Dynamic Pricing Engine:** Guides can set algorithmically tiered pricing (e.g., Solo Trekkers = ₹500, Group of 5 = ₹1500).
- Inventory Toggles: Local restaurants and water vendors can toggle items "In Stock" or "Out of Stock," instantly reflecting on the Trekker's screen.

3. Forest Rescue Command Radar (Safety Module) A highly restricted, low-latency dashboard designed exclusively for local authorities and rescue teams.

- **Distress Queue:** Intercepts incoming SOS payloads and renders the exact coordinates on a high-contrast topographical map.
- **Haptic Alerts:** Overrides device silent modes to trigger flashing UI elements and hardware vibration until the SOS is acknowledged by a dispatcher.

4. Root Admin Master Terminal (Overseer Module) The central "God Mode" web interface managing the entire MERN stack backend.

- **Ecosystem Analytics:** Aggregates total platform revenue, active user sessions, and database health.
- **Global Lockdown:** A critical toggle switch that instantly blocks all new bookings and pushes a "High Alert" banner to all active devices in case of severe weather.

3.2 Feasibility Study

Before writing the first line of code, a comprehensive feasibility study was conducted to ensure the proposed GadVata architecture was practically achievable within the constraints of a standard academic lifecycle. The project was evaluated across four critical dimensions: Technical, Economic, Operational, and Schedule feasibility.

Table 3.2.1: Feasibility Analysis Matrix

Feasibility Area	Analysis & Implementation Justification	Status
Technical Feasibility	Passed: The requirement for a cross-platform mobile app (Android/iOS) was solved using React Native and Expo, eliminating the need for two separate codebases. Node.js efficiently handles the asynchronous non-blocking API calls required for the real-time SOS and booking engines. MongoDB provides the flexible JSON-document structure needed for diverse multi-tenant profiles.	Highly Feasible
Economic Feasibility	Passed: The project utilizes predominantly open-source software and free-tier infrastructure. React Native, Node.js, and Express are open-source. External services like the Open-Meteo API (weather) and Expo Go (testing) are free. The primary cost is standard cloud database hosting (MongoDB Atlas), which falls well within an academic budget.	Highly Feasible
Operational Feasibility	Passed: The primary operational risk was user adoption among rural vendors. To mitigate this, the Partner Dashboard was designed with high-contrast, minimalist UI components (large toggle switches, simple forms) to ensure ease of use for individuals with minimal technical literacy.	Feasible

Schedule Feasibility	Passed: The project scale initially threatened the academic timeline. By adopting an Agile methodology and utilizing pre-built Expo modules	Feasible

3.3 Software Requirement Specification

The Software Requirement Specification (SRS) is the foundational blueprint of the system. It strictly defines what the system must do (Functional Requirements) and how the system must perform (Non-Functional Requirements).

3.3.1 Functional Requirements

Functional requirements define the specific behaviors, computational functions, and data manipulation tasks the GadVata platform must execute. The following table highlights the most critical functional workflows engineered into the system.

3.3.2 Functional Requirements Definition

Requirement ID	Target Module	Description of Required Functionality	Priority
FR-01	Authentication Gateway	The system must utilize JSON Web Tokens (JWT) or secure session hashing to authenticate all 6 user roles uniquely, routing them to their specific dashboards upon login.	High
FR-02	Real-Time Seat Matrix	Upon a successful trekker booking, the system backend must dynamically deduct the requested seats from the Agency's total inventory array and instantly update the UI.	High
FR-03	SOS Telemetry Dispatch	The system must extract the latitude and longitude from the device hardware, append the user's medical_string, and POST it to the Rescue node database bypassing standard calling.	Critical
FR-04	Offline Map Vault	The system must allow users to query and download PDF mapping buffers into the mobile device's	Medium

		local isolated storage for zero-network rendering.	
FR-05	Vendor State Toggle	The Partner dashboard must feature a binary toggle switch allowing local restaurants to change their store_status from true (Open) to false (Closed), instantly blocking new orders.	High
FR-06	Global Lockdown Protocol	The Root Admin terminal must feature a master boolean override. If set to true, the system must block all POST routes for new bookings platform-wide	Critical

3.3.2 Non-Functional Requirements

While functional requirements dictate what the GadVata system must do, non-functional requirements (NFRs) dictate *how well* the system must perform these tasks. These parameters are critical for ensuring the platform can handle real-world mountain conditions and high concurrent user loads without degrading or crashing.

1. Performance and Scalability

- **API Latency:** The system's backend (Node.js/Express) must process and return standard API requests (like fetching a fort's history) in under 200 milliseconds.
- **SOS Throughput:** The Emergency SOS broadcast must bypass standard queueing and be prioritized by the server, ensuring the payload hits the Rescue Command terminal in under 1 second of the trekker pressing the hardware trigger.
- **Concurrency:** The MongoDB cluster and Express server must be capable of handling at least 500 concurrent read/write operations per second, specifically to manage the live seat matrix during peak weekend booking times.

2. Security and Data Integrity

- **Authentication:** All user sessions (Trekkers, Vendors, Admins) must be secured using JSON Web Tokens (JWT) with strict expiration times. Passwords must be cryptographically hashed using the bcrypt library (minimum 10 salt rounds) before database insertion.
- **Role-Based Access Control (RBAC):** The system must strictly enforce API route protection. A Trekker token attempting to access a Vendor inventory POST route must be immediately rejected with a 403 Forbidden status.

- **Data Encryption:** All sensitive medical data (e.g., blood type, severe allergies) attached to the SOS payload must be encrypted in transit using HTTPS/TLS 1.2 protocols.

3. Reliability and Availability

- **Offline Resilience:** The frontend mobile application (React Native) must cache critical state data using AsyncStorage. If the user loses network connection, the app must not crash but instead gracefully load the cached data and offline PDF tactical maps.
- **Uptime:** The backend server and database must maintain a 99.9% uptime requirement during the primary trekking season (monsoon and winter).

4. Usability

- **Cross-Platform Parity:** The user interface must render uniformly across both Android and iOS devices, accommodating different screen notches and aspect ratios using React Native's SafeAreaView.
- **High-Contrast Visibility:** Given the outdoor usage of the app, the UI must support a dynamic Light/Dark theme engine with high-contrast text to ensure readability under direct mountain sunlight.

3.4 System software and Hardware requirements

To successfully engineer, test, and deploy the GadVata platform, a specific stack of hardware and software was utilized. The development environment required robust processing power for running local Android/iOS emulators simultaneously with the Node.js backend.

The following table details the exact specifications utilized during the 2025-2026 academic development cycle

Category	Specification / Technology Utilized
Development Hardware (Processor)	Intel Core i5 (10th Gen) / AMD Ryzen 5 / Apple Silicon M1
Development Hardware (Memory)	16 GB RAM (Minimum requirement for running React Native bundlers and emulators concurrently)
Development Hardware (Storage)	512 GB SSD (For high-speed read/write during Node module installation)
Testing Hardware (Mobile)	Standard Android Device (OS Version 10.0+) and iOS Device (iPhone 17) for native camera and GPS hardware testing.
Frontend Software Stack	React Native (v0.72+), Expo CLI, React Navigation, Axios (for API requests)
Backend Software Stack	Node.js (v18.0+), Express.js framework, JSON Web Tokens (JWT) for Auth.
Database Architecture	MongoDB Atlas (Cloud NoSQL Cluster), Mongoose (Object Data Modeling library).
Development Environment (IDE)	Visual Studio Code (VS Code) with Prettier and ESLint formatting extensions.
Version Control & Hosting	Git, GitHub (Repository Management)

3.5 Software Development Life Cycle (SDLC) Model

For the development of the GadVata Evolution ecosystem, I explicitly chose to follow the **Agile Iterative Model**. I chose Agile because building a multi-tenant application with hardware integrations (like GPS and Haptic SOS) is highly complex. The project requirements evolved dynamically as I tested the app, and Agile allowed me to adapt the codebase based on real-world testing rather than getting stuck in a rigid, traditional Waterfall process.

1. Requirement Gathering and Analysis

- In this initial phase, I identified the severe infrastructural pain points of mountain trekking, specifically the lack of centralized route data, offline maps, and the delayed emergency response times.
- I defined the core interconnected modules: The Trekker Application (B2C), the Vendor/Partner Dashboard (B2B), the Forest Rescue Radar, and the Root Admin Master Terminal.

2. System Design

- I created the System Architecture and ER Diagrams to map out exactly how my MongoDB NoSQL database would handle distinct schemas for six different user roles (Trekkers vs. Vendors).
- The UI/UX was designed during this stage in Figma, focusing heavily on high-contrast toggle switches for local vendors and a strictly minimal, one-tap UI for the Emergency SOS broadcast button.

3. Implementation (Coding)

- Development was broken down into two-week "Sprints." My early sprints focused on establishing the React Native frontend and the secure JWT Authentication gateway.
- Later sprints were highly technical, where I used Node.js and Express to build the dynamic live seat matrix engine and integrated the expo-location hardware APIs for real-time telemetry extraction.

4. Testing

- **Iterative Testing:** Because GadVata relies heavily on physical hardware (GPS coordinates, vibration motors), I could not rely solely on web emulators. After every feature update, I tested the React Native build directly on physical Android and iOS devices using Expo Go.

- **User Acceptance Testing (UAT):** I rigorously tested the Partner panel to ensure that a non-technical local vendor could easily toggle a food item from "In Stock" to "Out of Stock" and see it update instantly on the Trekker's screen.

5. Deployment

- The mobile application frontend was deployed and managed via the Expo ecosystem, ensuring seamless cross-platform functionality.
- The Node.js backend and routing infrastructure were deployed to cloud hosting, actively connected to the live MongoDB Atlas cloud cluster.

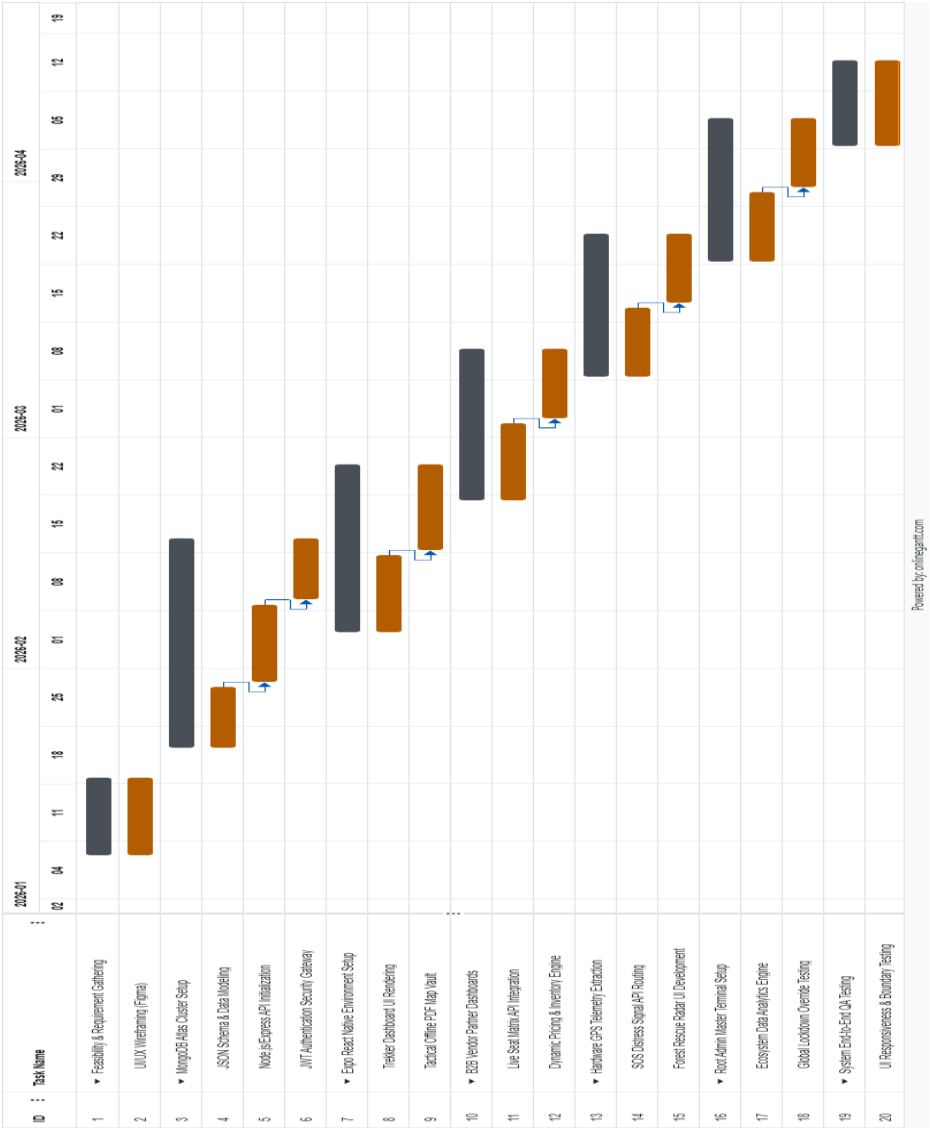
6. Maintenance and Refinement

- Based on continuous API testing, I heavily refined the SOS payload structure. I optimized the database queries to ensure that a user's exact geolocation and pre-filled medical string were captured and routed to the Rescue node in under one second, proving the system's life-saving reliability.

3.6 Project Development Scheduling

Task ID	Development Focus	Start Date	Completed
1.0	Feasibility & Requirement Gathering	05 Jan 2026	10 Jan 2026
1.1	UI/UX Wireframing (Figma)	09 Jan 2026	18 Jan 2026
2.0	MongoDB Atlas Cluster Setup	19 Jan 2026	24 Jan 2026
2.1	JSON Schema & Data Modeling	22 Jan 2026	29 Jan 2026
2.2	Node.js/Express API Initialization	26 Jan 2026	04 Feb 2026
2.3	JWT Authentication Security Gateway	01 Feb 2026	08 Feb 2026
3.0	Expo React Native Environment Setup.	02 Feb 2026	06 Feb 2026
3.1	Trekker Dashboard UI Rendering	05 Feb 2026	14 Feb 2026
3.2	Tactical Offline PDF Map Vault	12 Feb 2026	22 Feb 2026
4.0	B2B Vendor Partner Dashboards	16 Feb 2026	24 Feb 2026
4.1	Live Seat Matrix API Integration	21 Feb 2026	02 Mar 2026
4.2	Dynamic Pricing & Inventory Engine	28 Feb 2026	08 Mar 2026
5.0	Hardware GPS Telemetry Extraction	02 Mar 2026	10 Mar 2026
5.1	SOS Distress Signal API Routing	08 Mar 2026	16 Mar 2026
5.2	Forest Rescue Radar UI Development	14 Mar 2026	22 Mar 2026
6.0	Root Admin Master Terminal Setup	16 Mar 2026	24 Mar 2026
6.1	Ecosystem Data Analytics Engine	22 Mar 2026	30 Mar 2026
6.2	Global Lockdown Override Testing	28 Mar 2026	05 Apr 2026
7.0	System End-to-End QA Testing	30 Mar 2026	12 Apr 2026
7.1	UI Responsiveness & Boundary Testing	08 Apr 2026	26 Apr 2026

Gannt Chart – GadVata : Fort Exploration Engine



Chapter 4. System Design

This chapter translates the software requirements identified in Chapter 3 into a concrete technical blueprint. It details the system architecture of the GadVata Evolution platform and utilizes standard Unified Modeling Language (UML) diagrams—including Use Case Diagrams, Data Flow Diagrams (DFDs), and Entity-Relationship Diagrams (ERDs)—to visually map the data flow and user interactions across the ecosystem.

4.1 System Architecture

The GadVata platform employs a highly scalable, decoupled Client-Server architecture utilizing the MERN stack integrated with React Native. The architecture is explicitly designed to handle asynchronous multi-tenant data streams (e.g., simultaneous trekker bookings and vendor inventory updates) without blocking the main execution thread.
The architecture is divided into three primary tiers:

1. **Presentation Tier (Frontend):** Built utilizing React Native and Expo. This tier processes native hardware requests (capturing GPS hardware coordinates via expo-location and local storage caching for offline PDF maps). It communicates with the backend exclusively via secure HTTPS REST APIs.

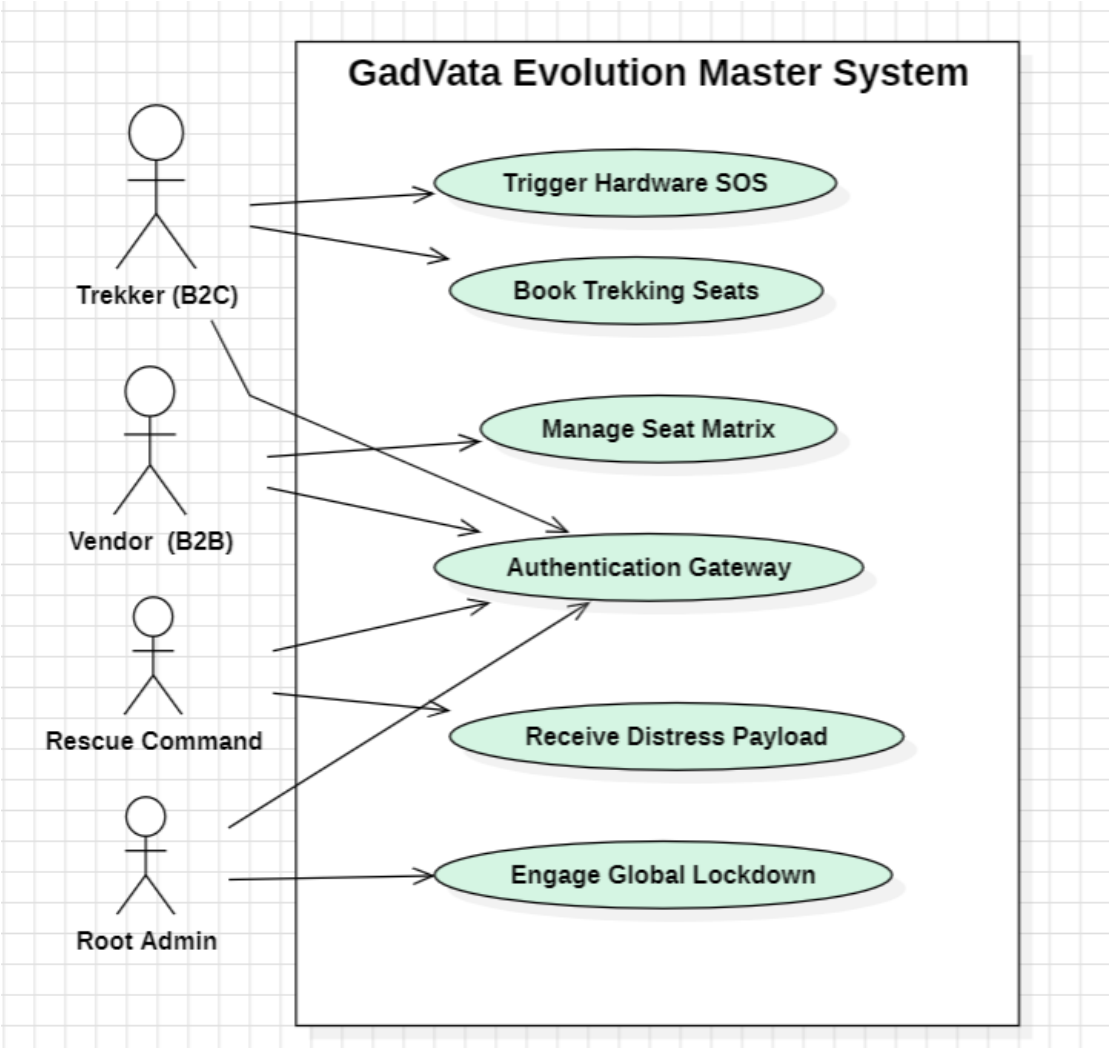
2. **Logic Tier (API Gateway & Backend):** Hosted on a Node.js runtime utilizing the Express.js framework. This tier acts as the central nervous system. It contains the core algorithmic logic, including the JWT authentication middleware, the dynamic seat matrix deduction engine, and the low-latency SOS distress router.

3. **Data Tier (Database):** A cloud-hosted MongoDB Atlas NoSQL cluster. This tier stores the multi-tenant JSON/BSON schemas. NoSQL was chosen over standard SQL to allow for highly flexible document structures (e.g., nesting varied medical history arrays within a trekker's profile).

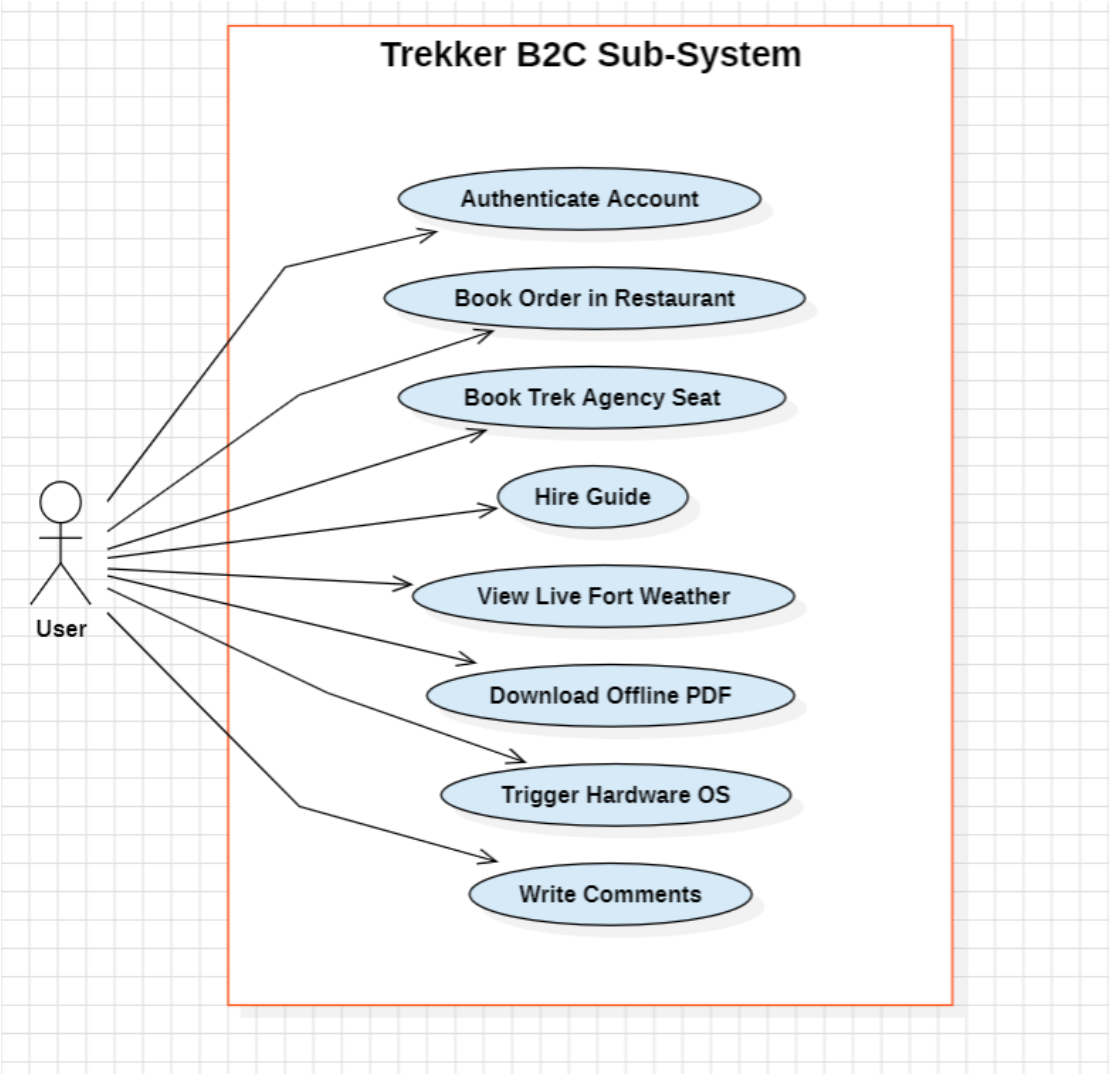
4.2 Use Case Analysis

Use Case diagrams are utilized to define the functional boundaries of the system from the perspective of the end-users (Actors). The GadVata ecosystem operates with four primary functional actors, each possessing strictly defined privilege levels governed by Role-Based Access Control (RBAC).

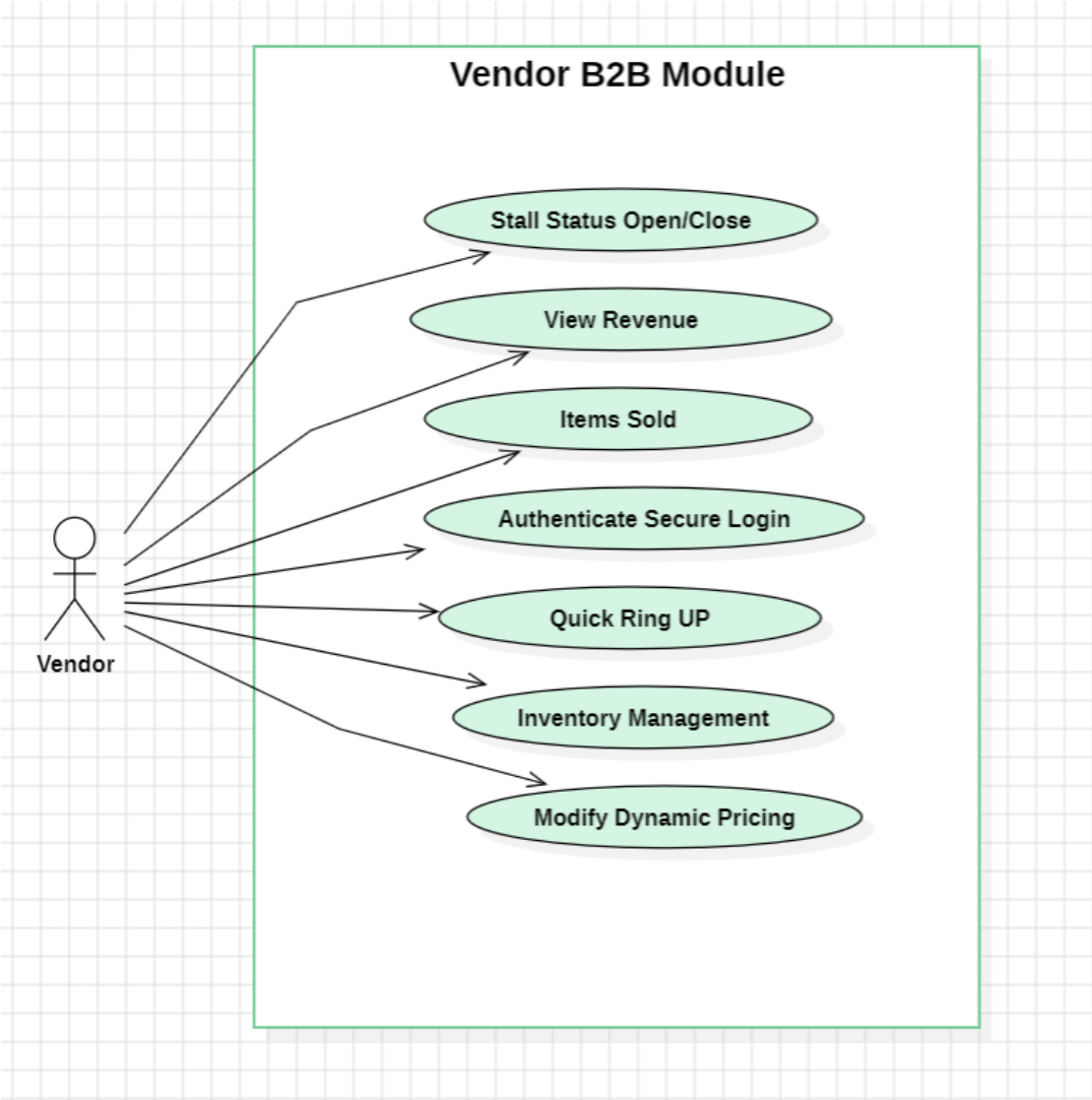
4.2.1 GadVata Evolution Master System



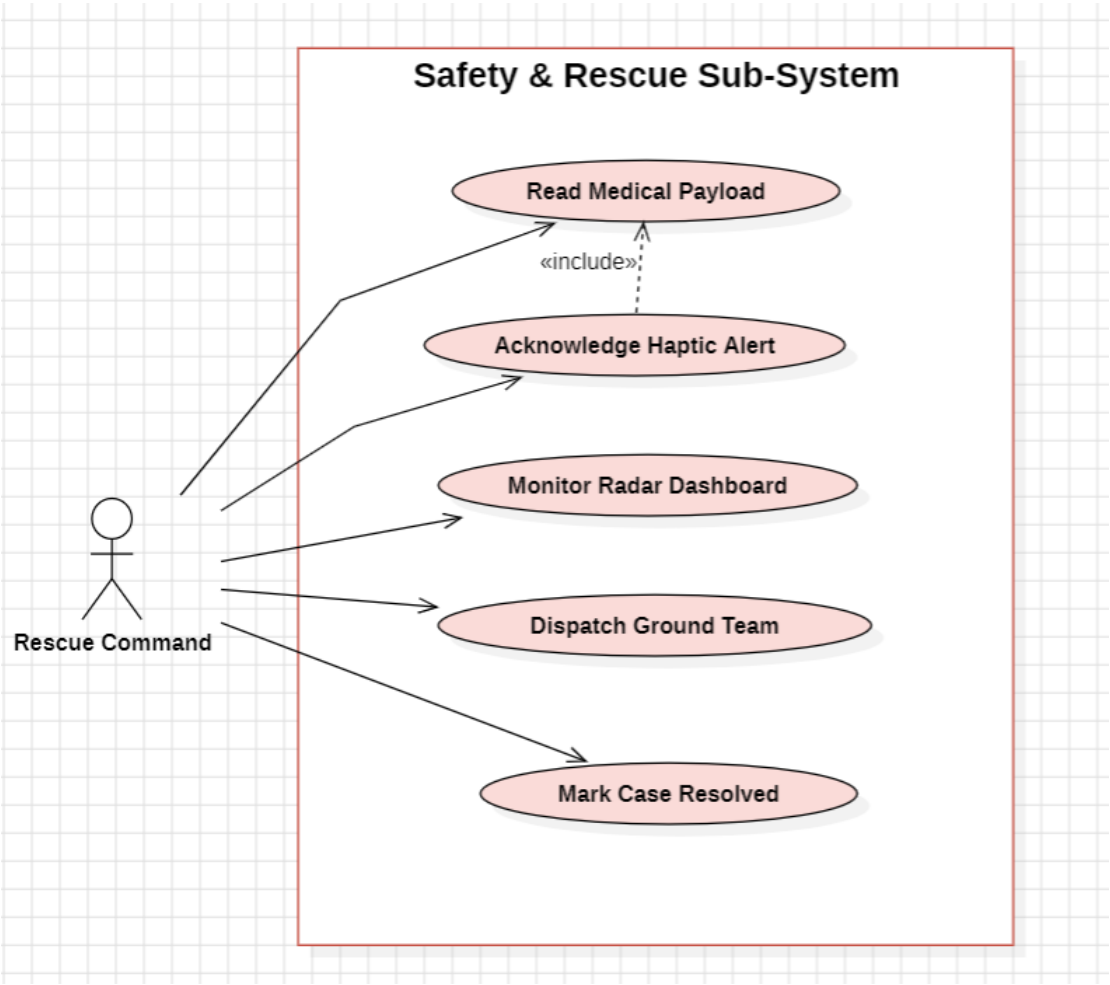
4.2.2 Trekker B2C Sub-System



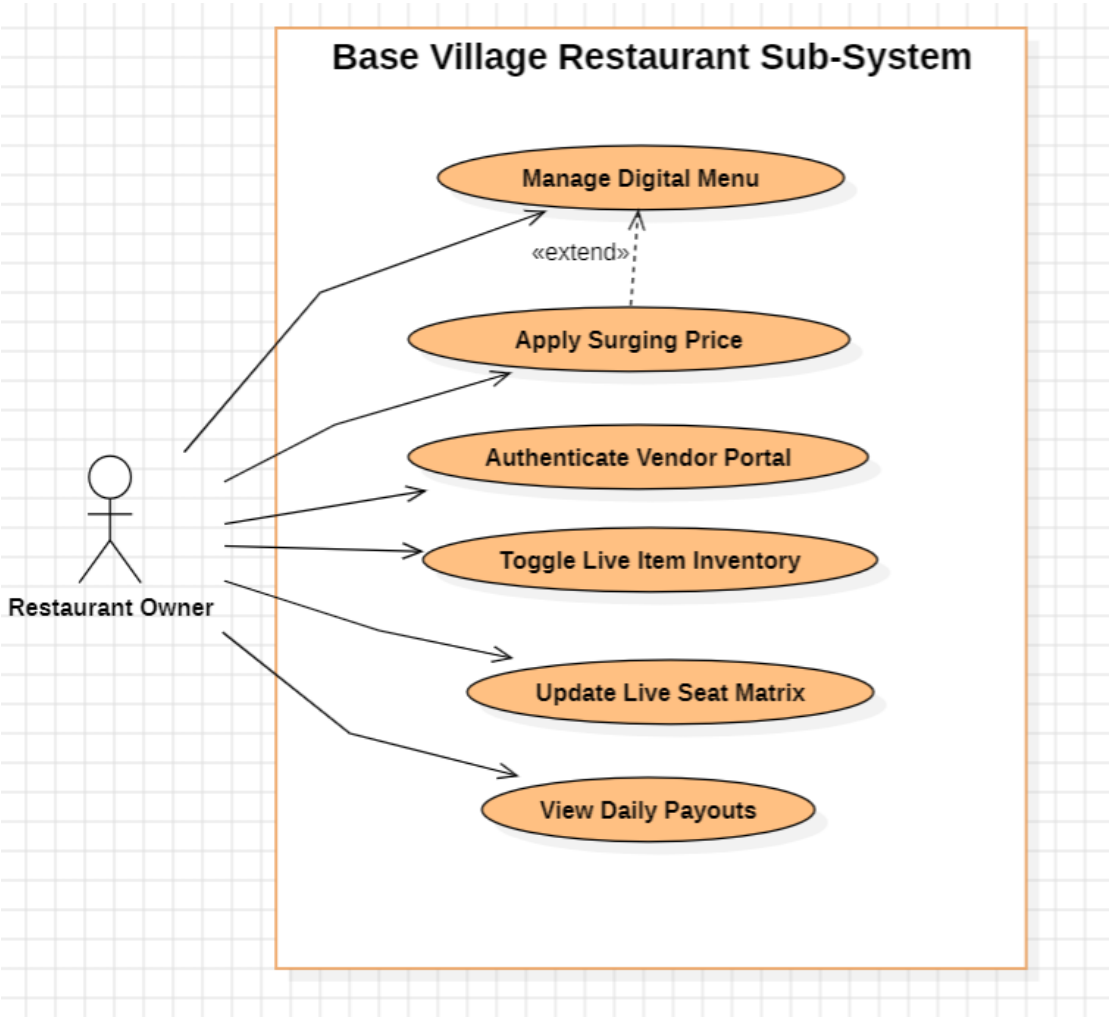
4.2.3 Vendor B2B Module



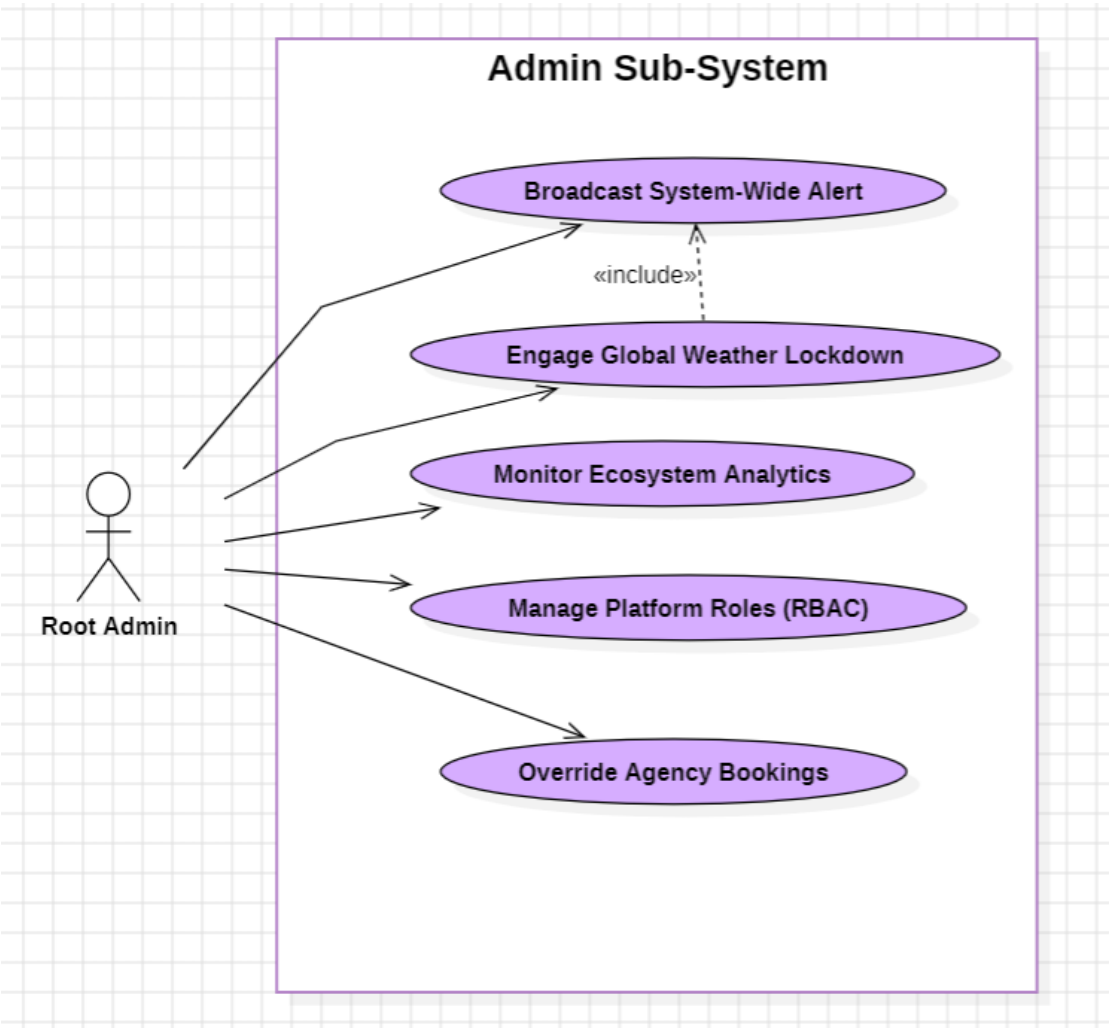
4.2.4 Safety & Rescue Sub-System



4.2.5 Base Village Restaurant Sub-System

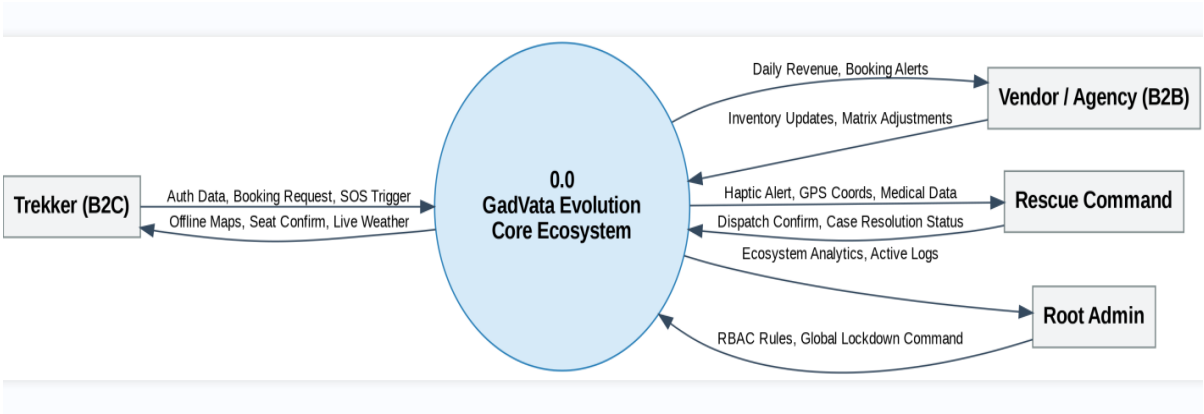


4.2.6 Admin Sub-System

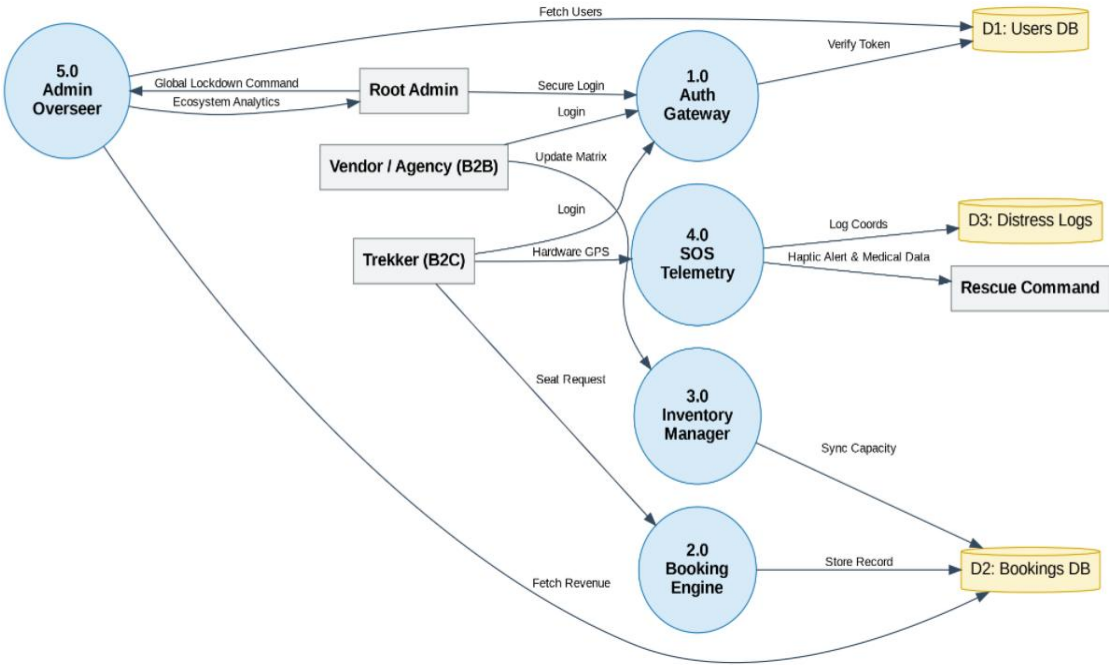


4.2 Data Flow Diagram

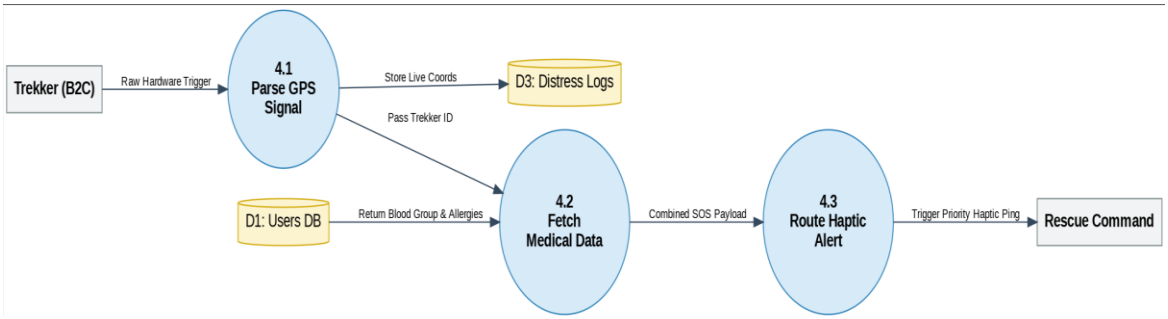
1) Level O DFD



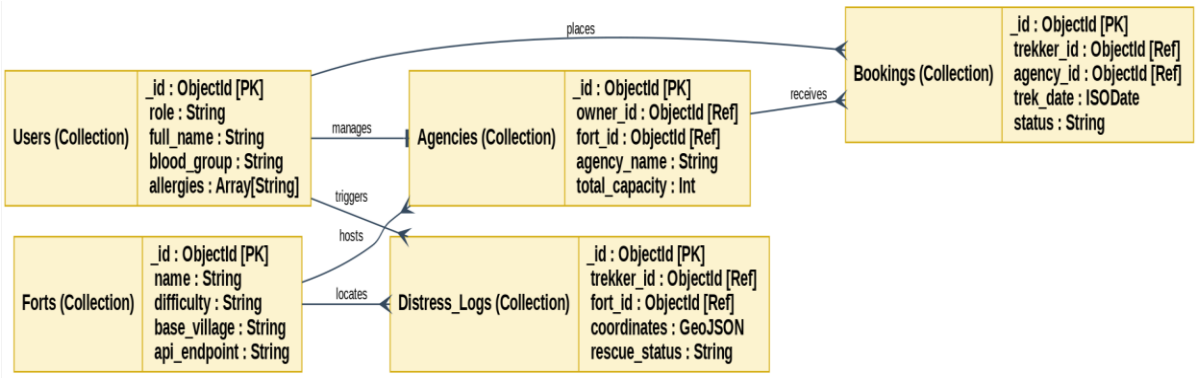
2) Level 1 DFD



3) Level 2 DFD

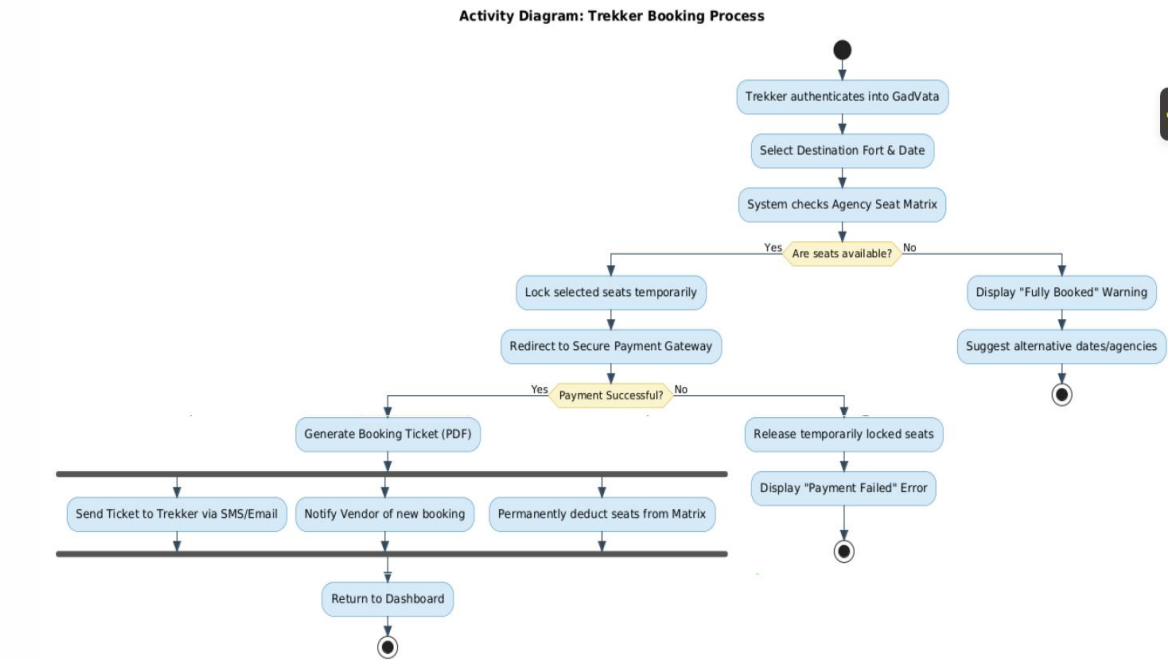


4.3 ER Diagram

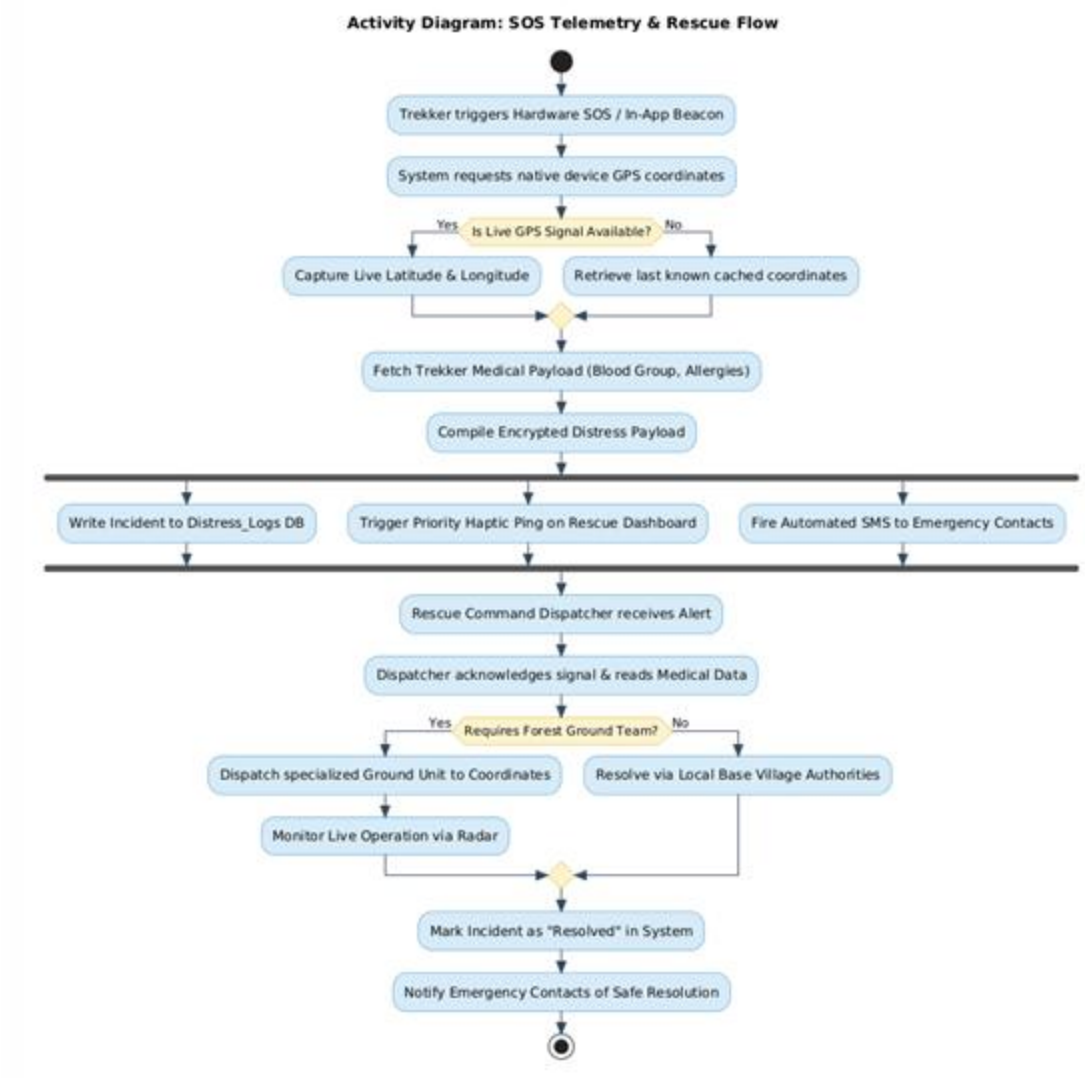


4.4 Activity Diagram

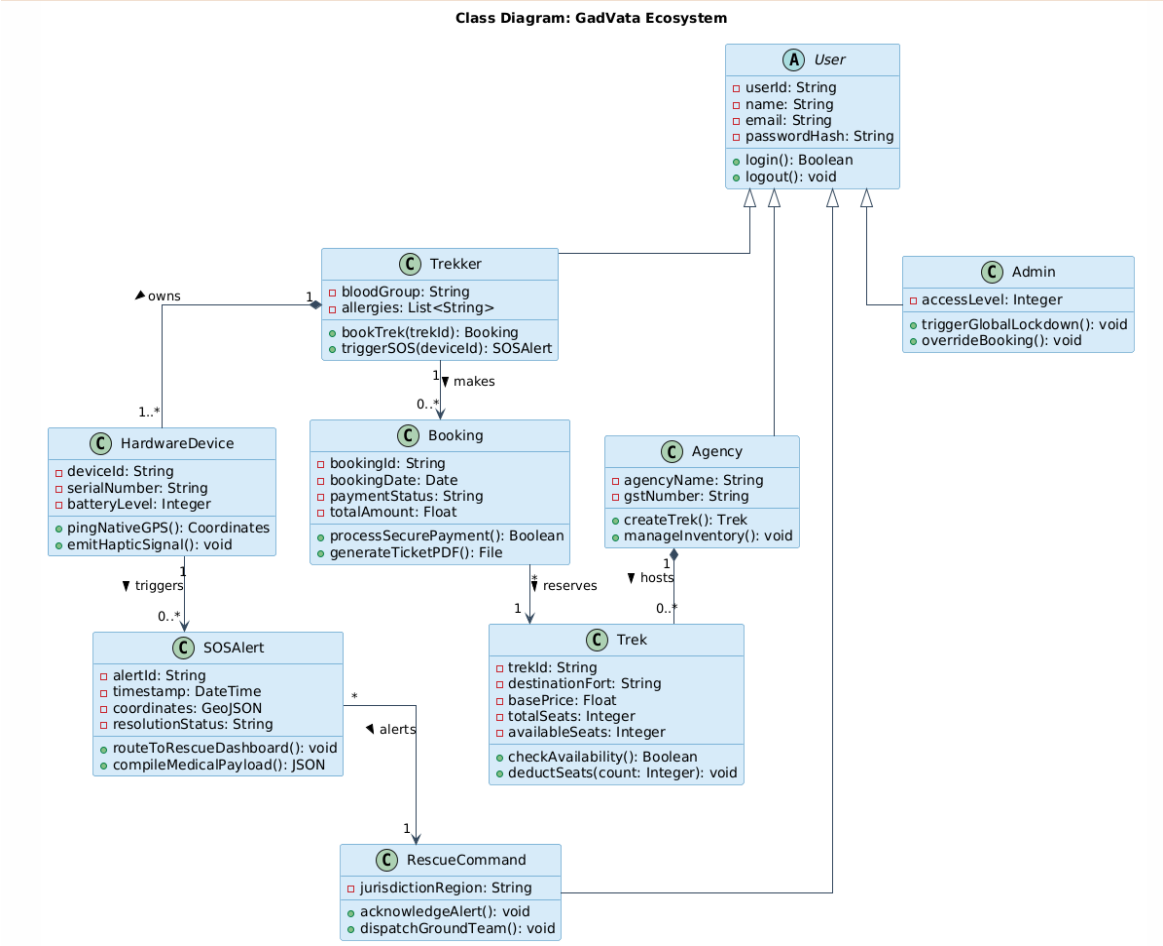
4.4.1 Trekker Booking Process



4.4.2 SOS Rescue Flow

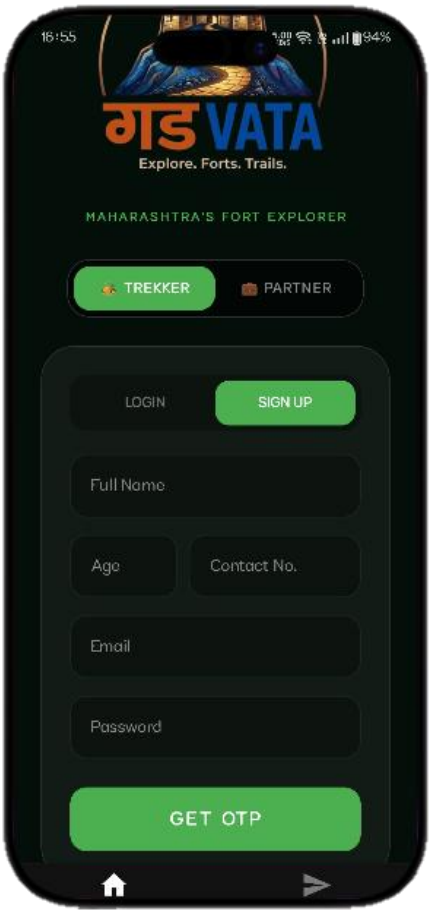


4.5 Class Diagram

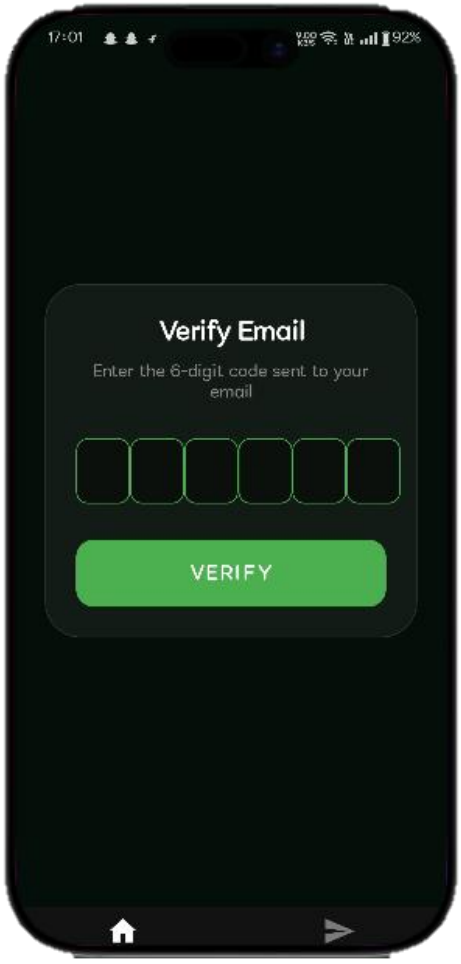


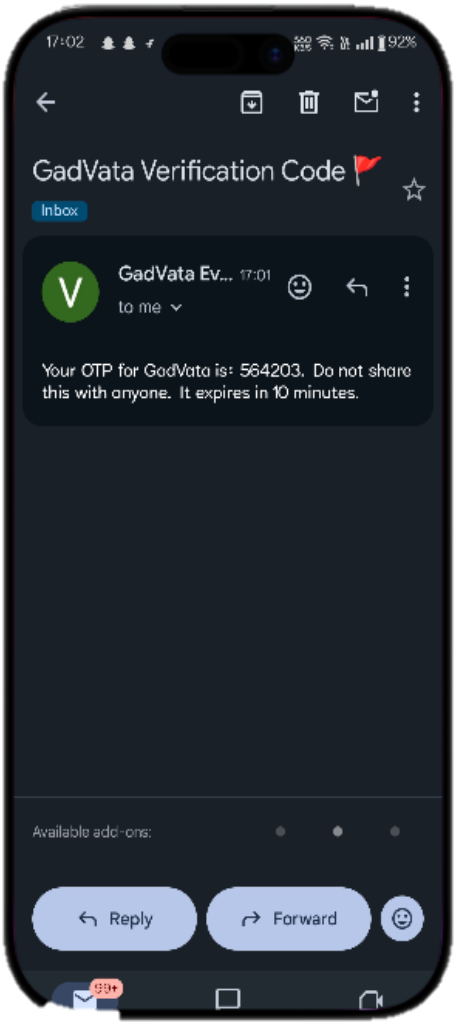
4.5 Project System Screens

4.5.1) GadVata Login / SignUp



4.5.2 Otp Verification

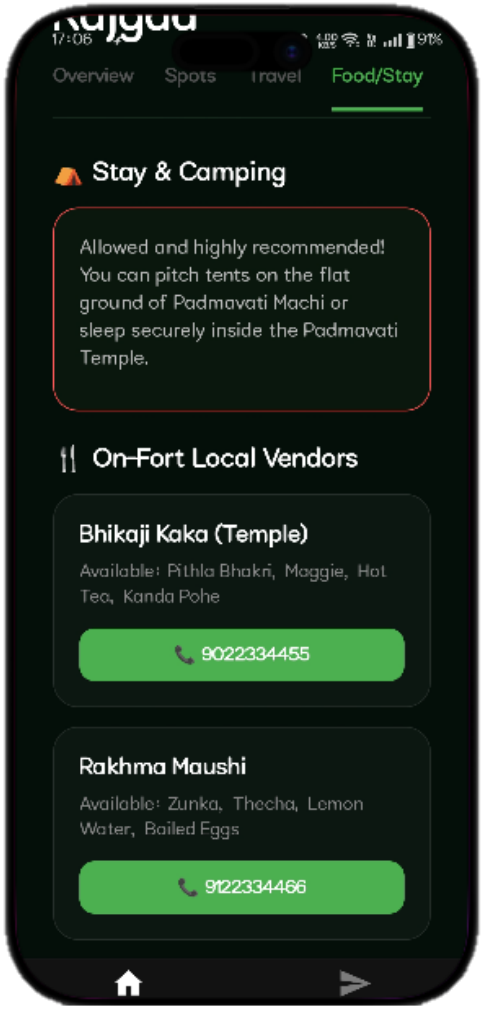




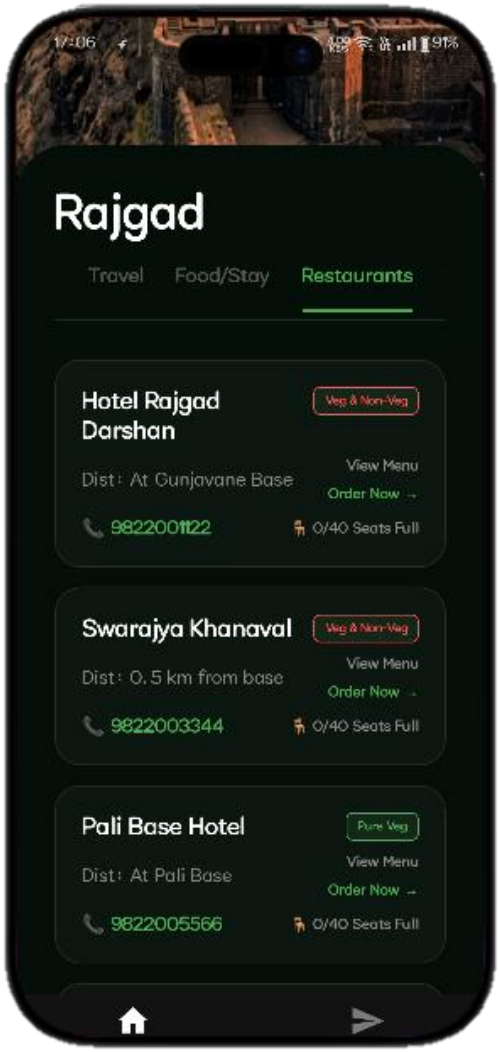
4.5.3) Home Screen

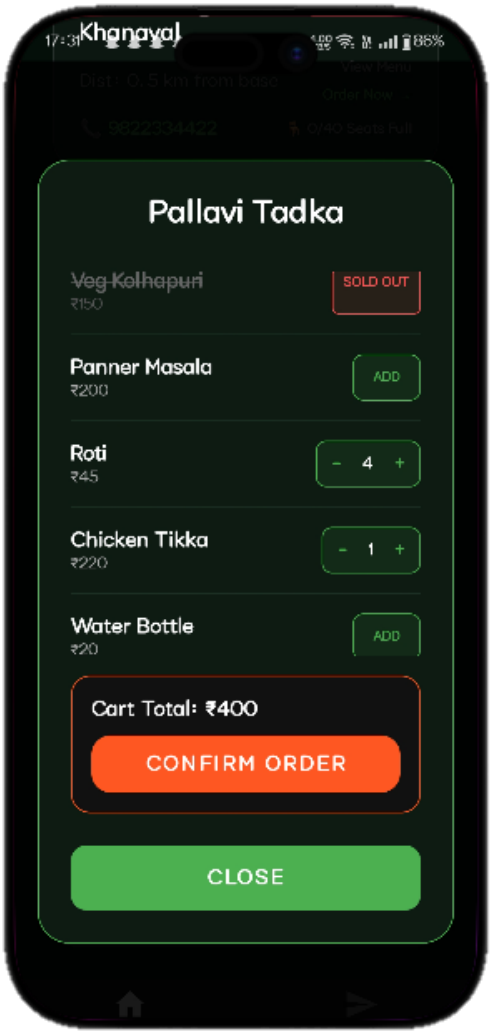
4.5.4) Overview & Spots

4.5.5) Travel & Food/Stay

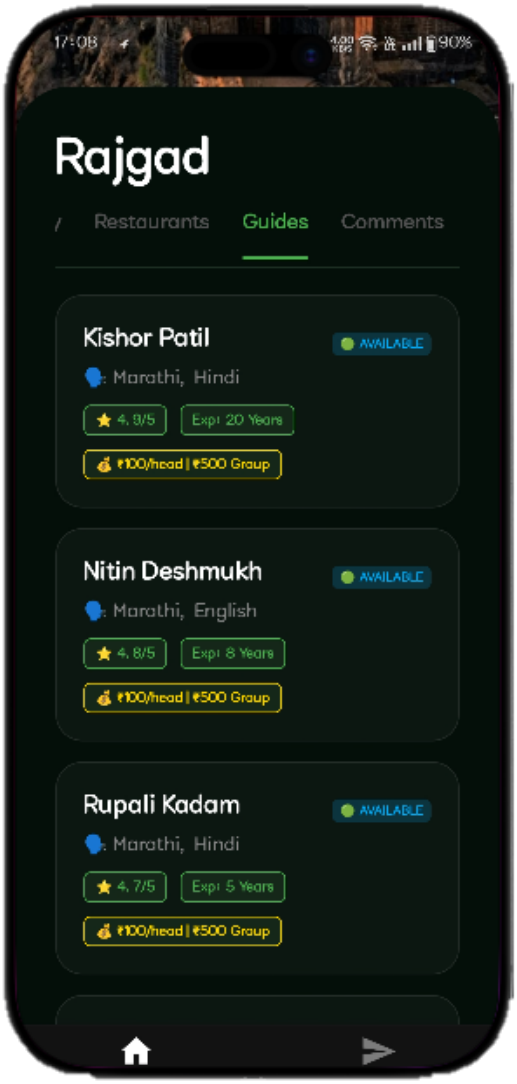


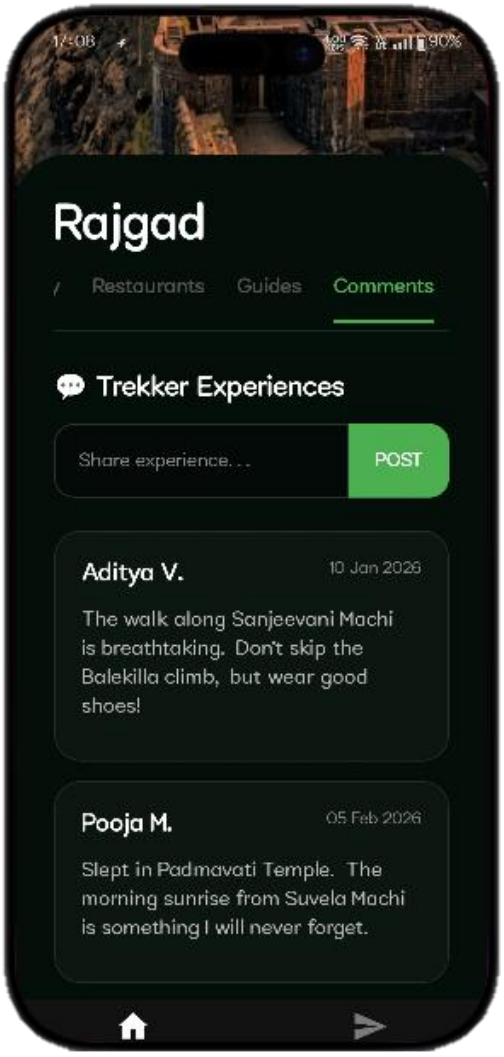
4.5.6) Restaurants



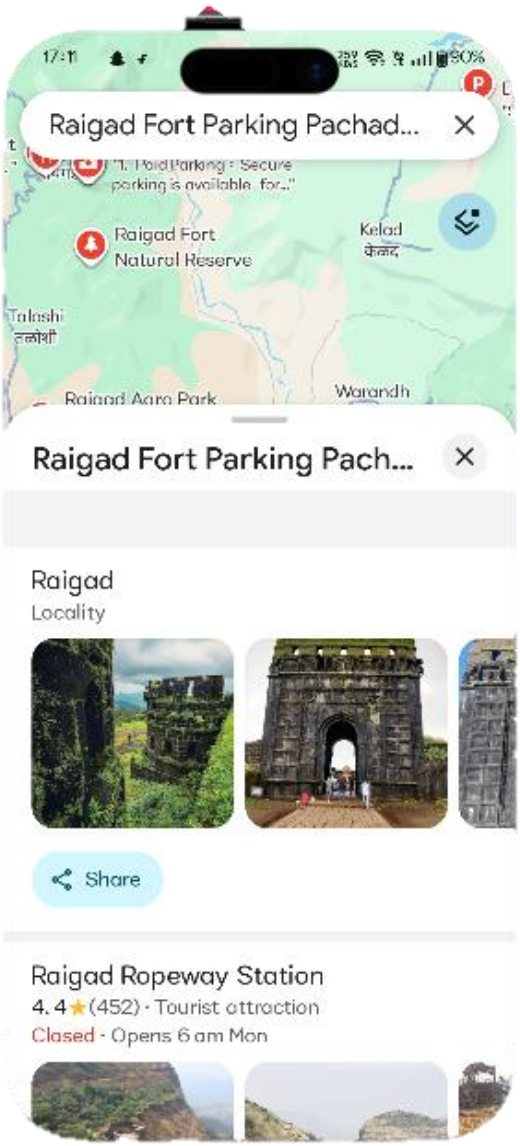


4.5.7) Guides & Comments



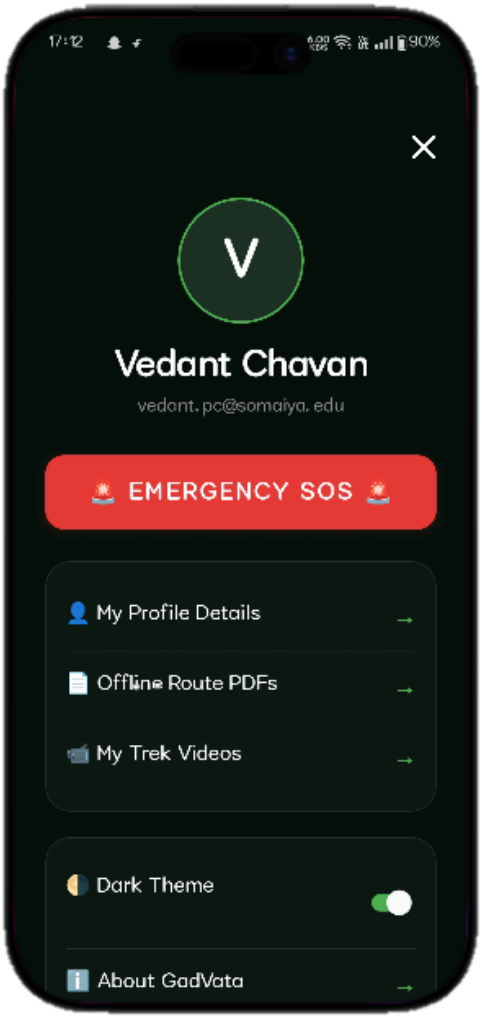


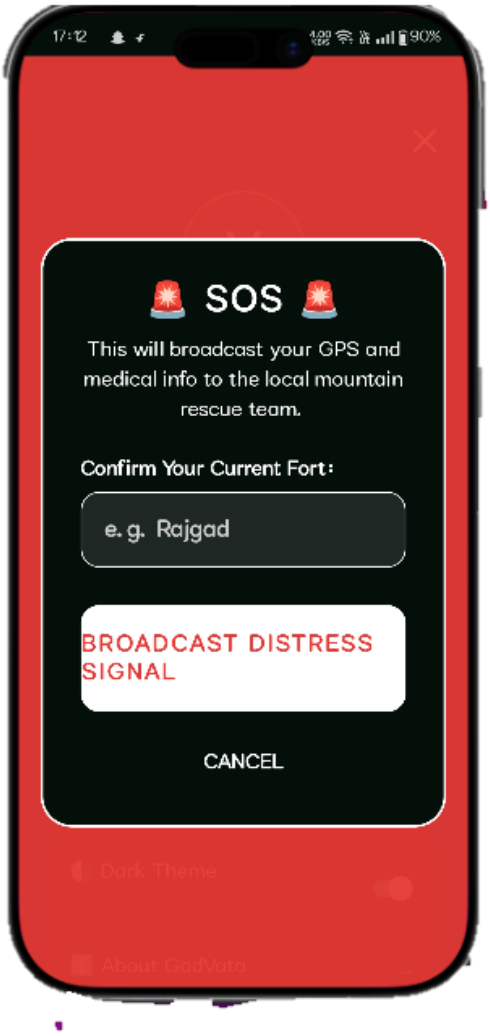
4.5.8) Navigation Tab



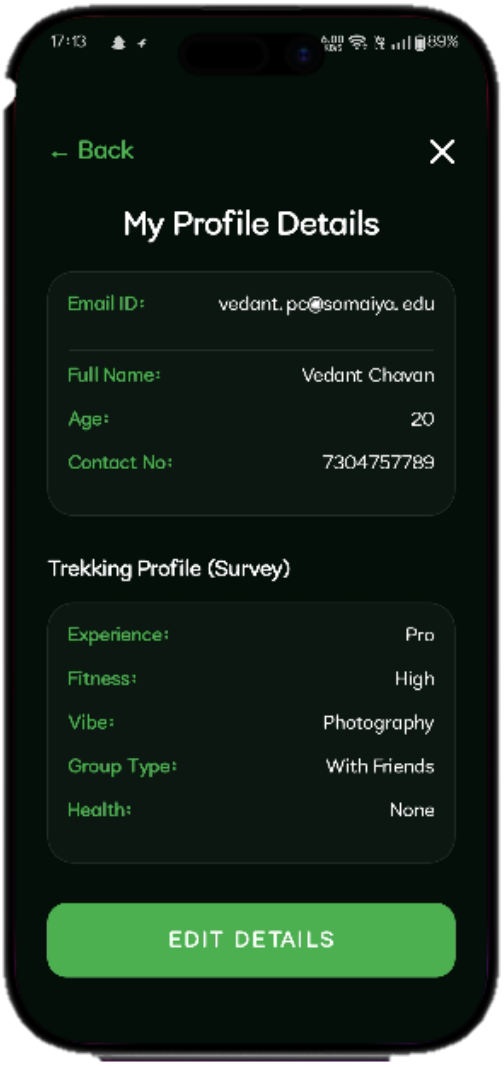
4.5.9) Trails Tab

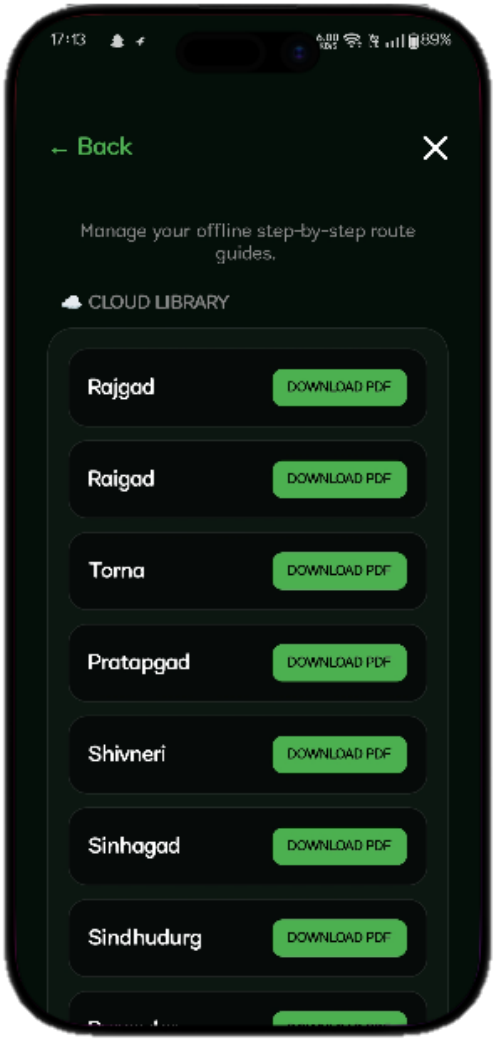
4.5.10) Settings Tab



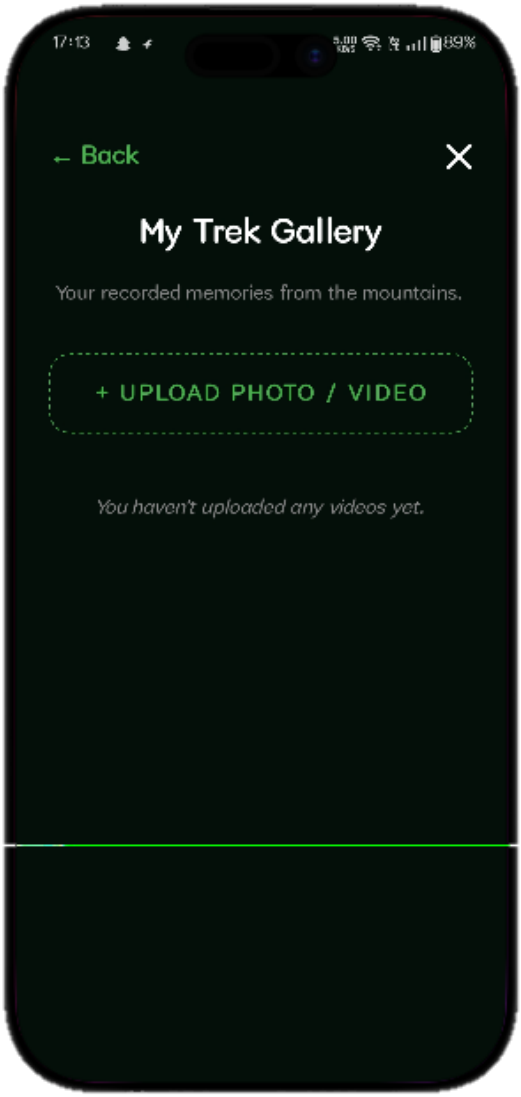


4.5.11) Profile Details / PDF

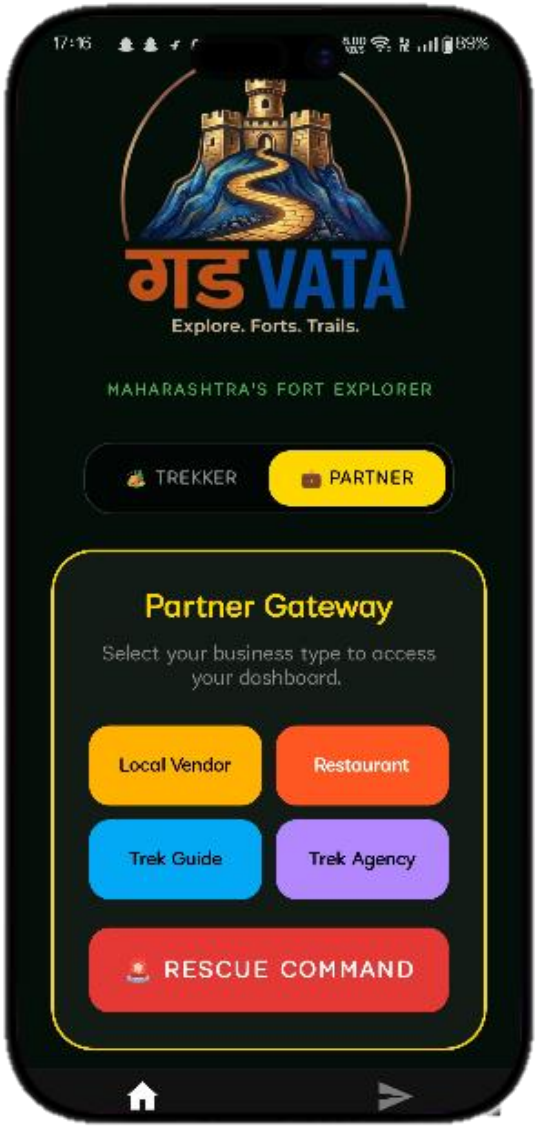


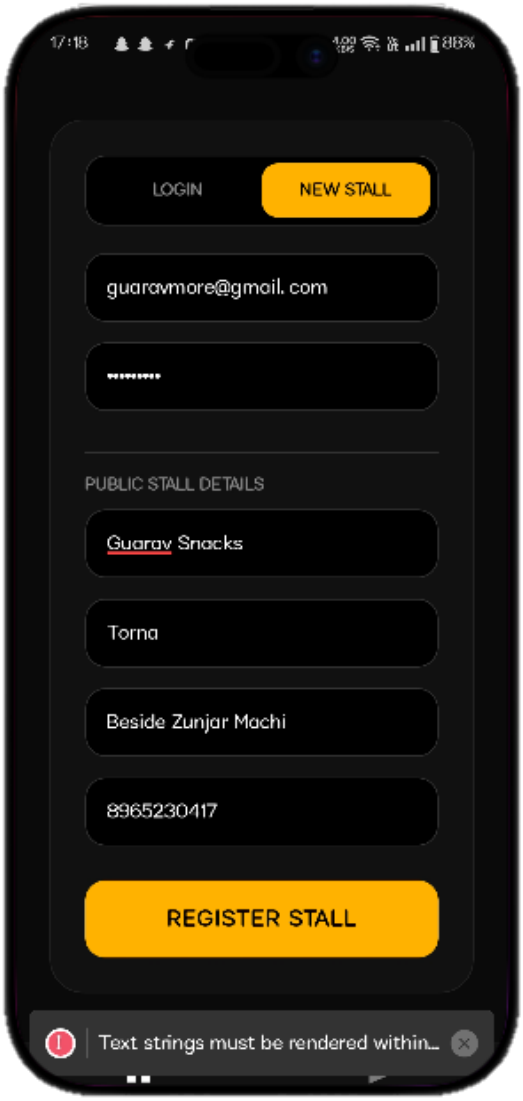


4.5.12) Trek Gallery

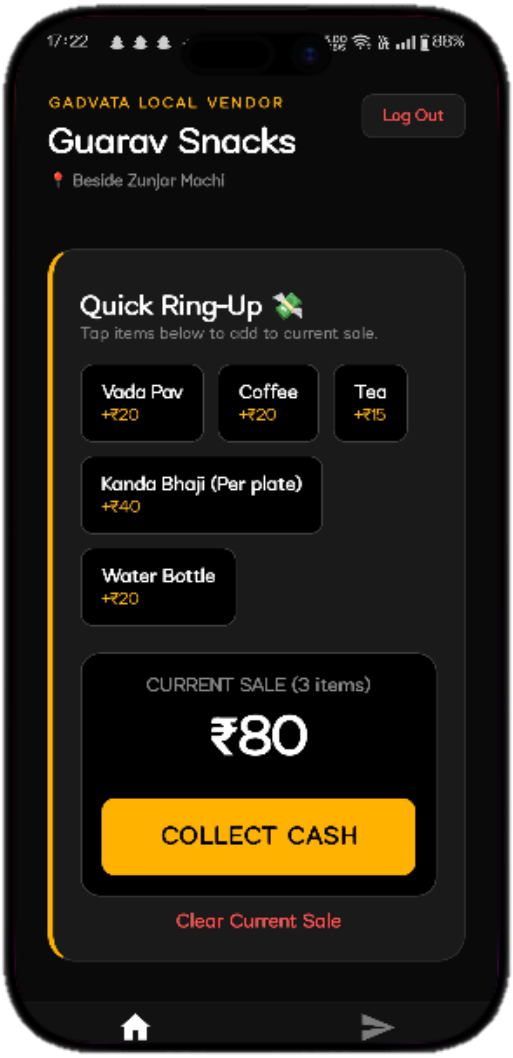


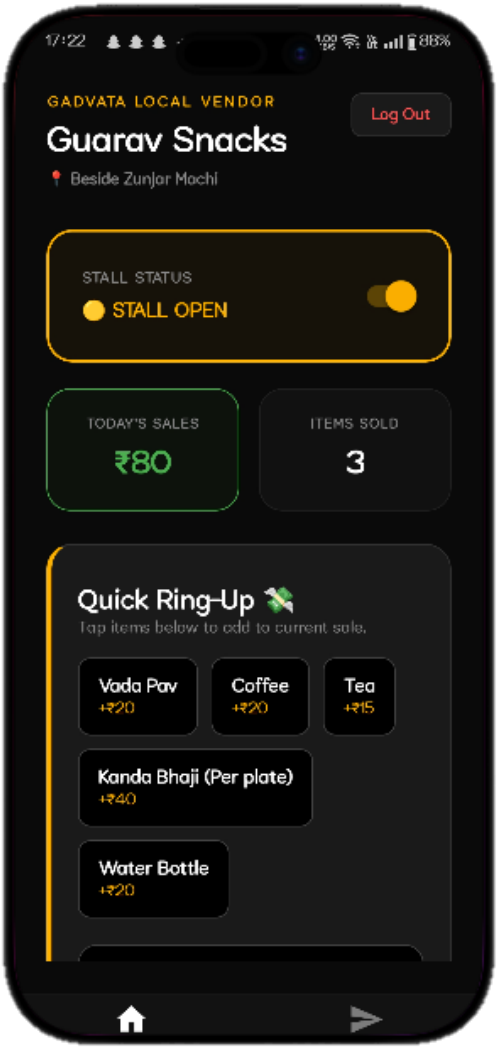
4.5.13) GadVata Partner Mode



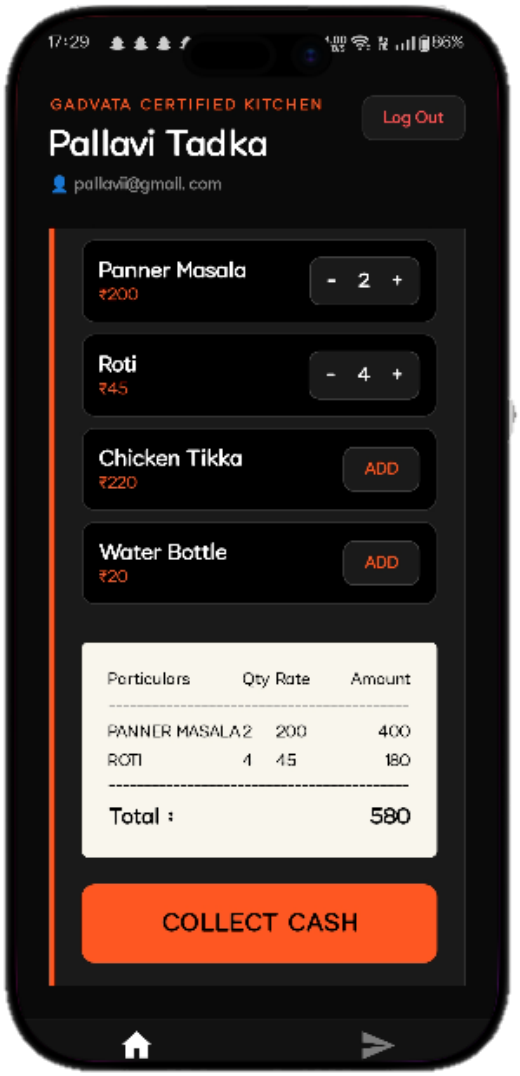


4.5.13) Local Vendor

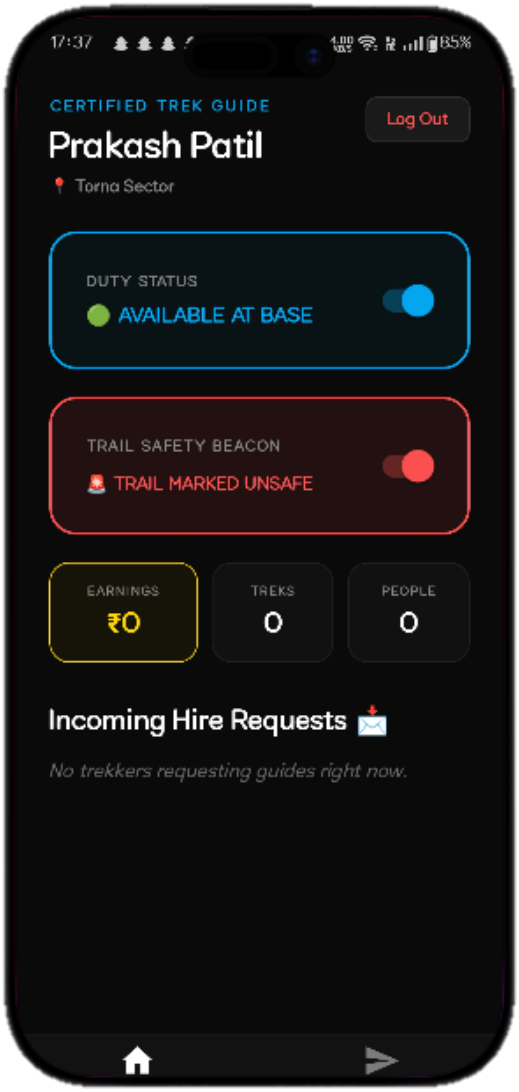




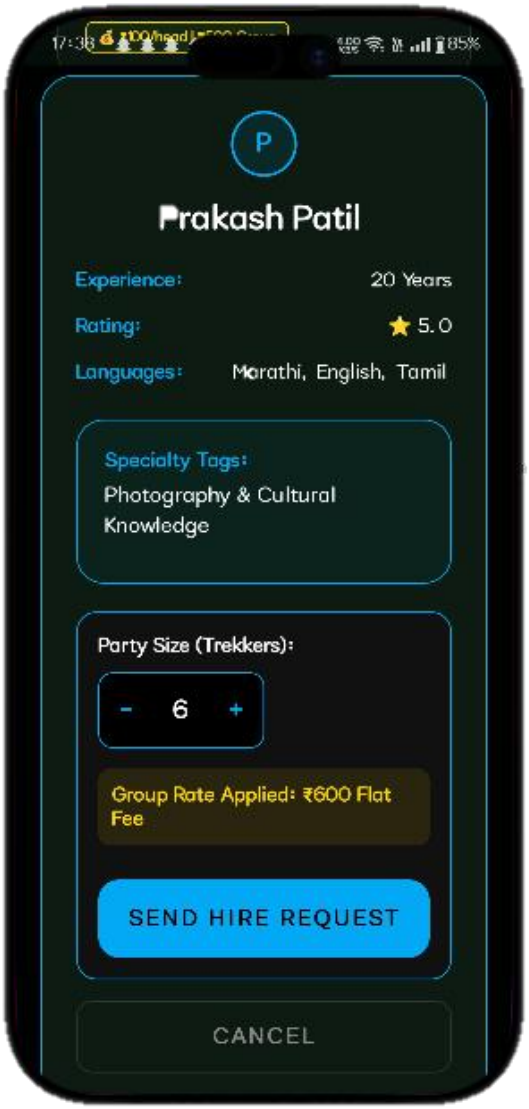
4.5.14) Restaurant

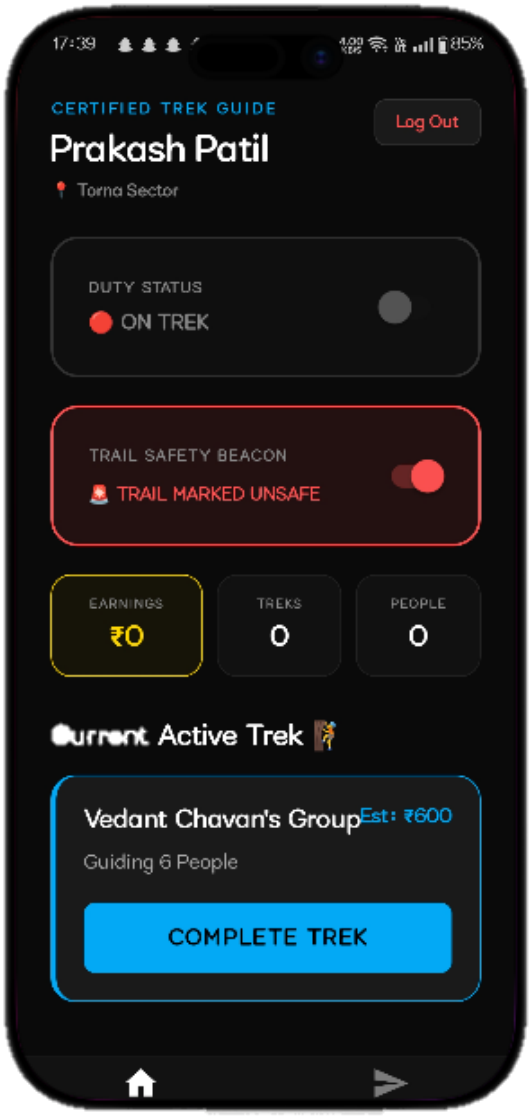


4.5.15) Trek Guides

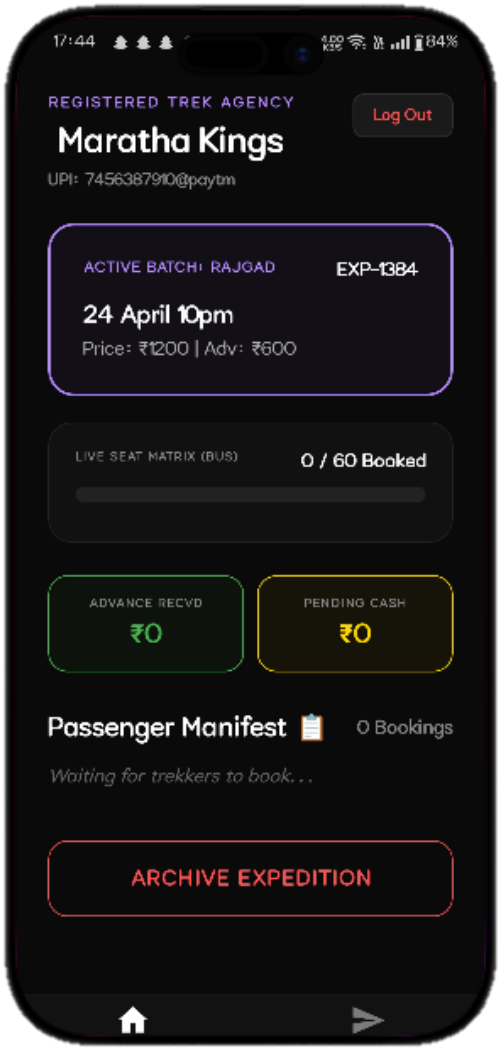


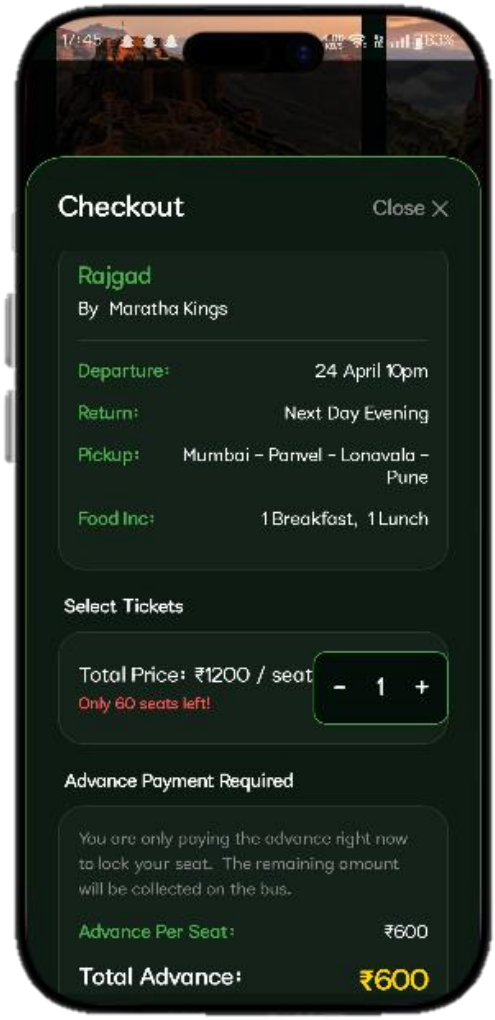
4.5.16) Guides Profile / Incoming Request



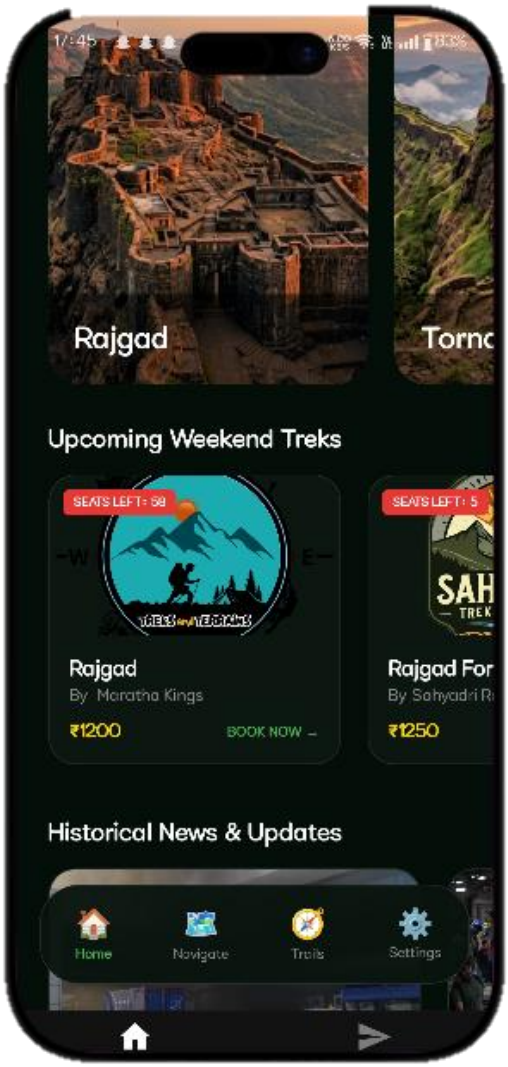


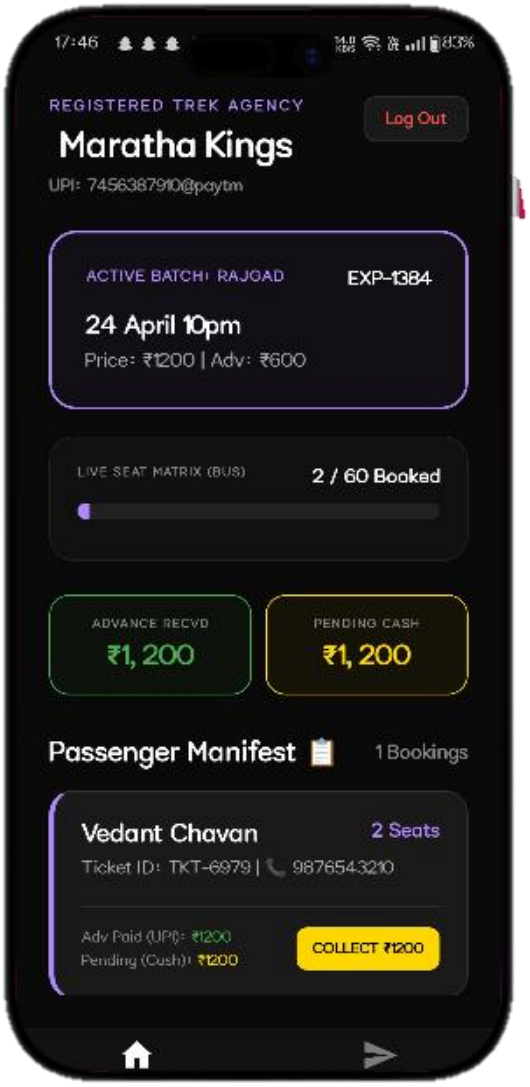
4.5.17) Trek Agency



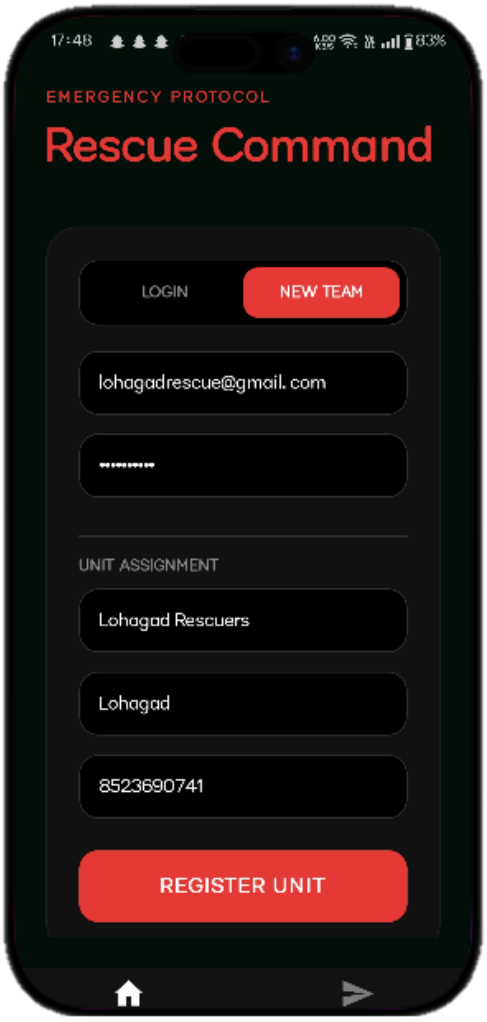


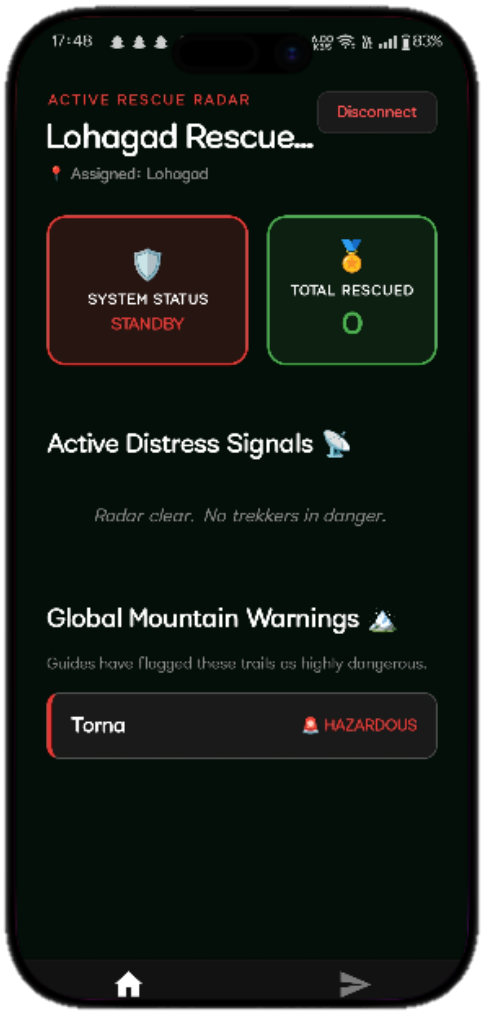
4.5.18) New Trek Agency Added / Advance Payment



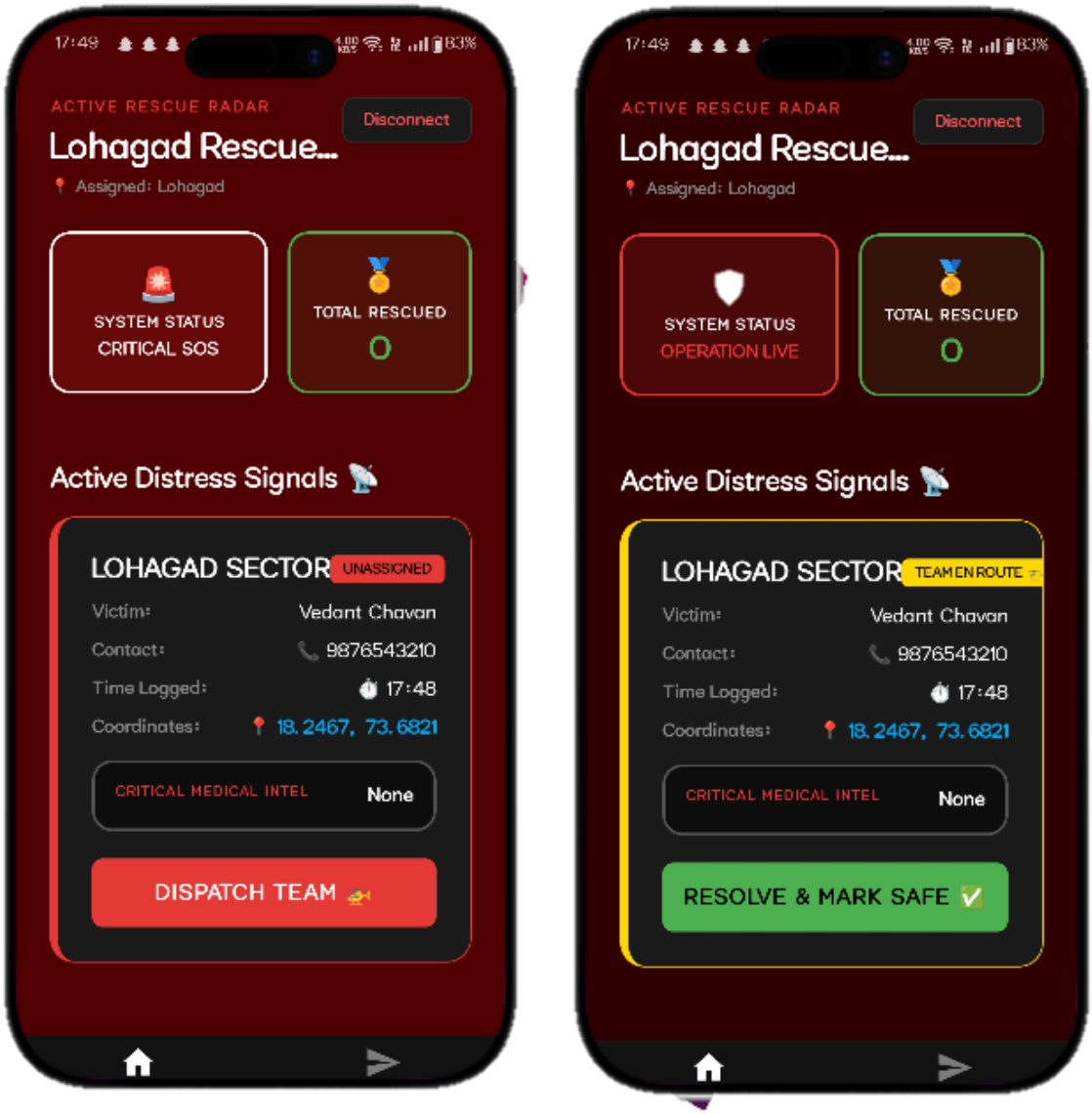


4.5.19) Rescue Command

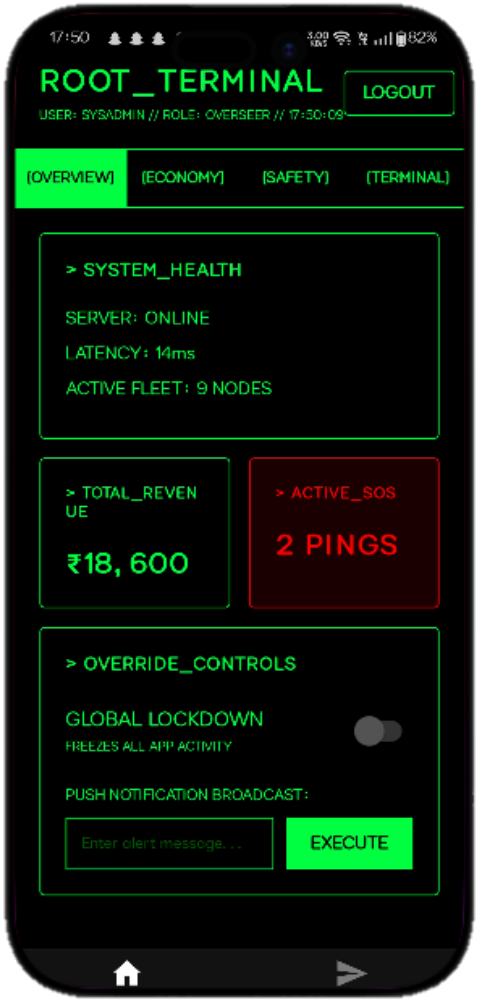


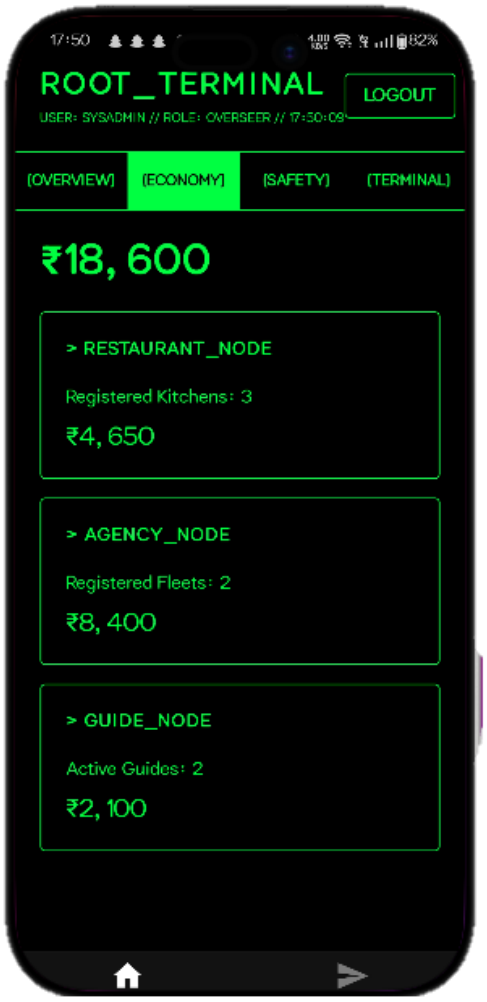


4.5.20) Distress Signals

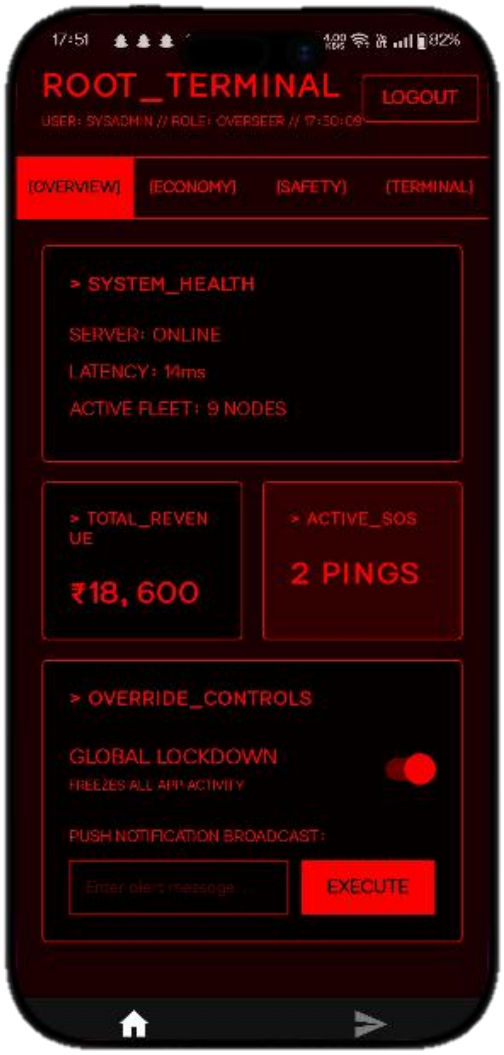


4.5.21) Admin Page





4.5.22) Admin Page Terminal / Global Lockdown



Chapter 5 : Implementation & Testing

5.1 Algorithms and Major Modules of the Project

The implementation phase translates the architectural blueprints of the GadVata ecosystem into a functional software application. The development is modular, allowing for independent execution and scaling.

Major Modules of GadVata:

1. **B2C Trekker Module:** Manages user authentication, digital fort exploration, offline map caching, and secure seat booking.
2. **B2B Vendor & Agency Module:** Provides trek agencies and local restaurants with a dashboard for live seat matrix updates, inventory toggling, and cash-collection tracking.
3. **Rescue Command (SOS) Module:** A critical safety telemetry engine that captures hardware GPS triggers, compiles medical payloads, and routes alerts to dispatchers.
4. **Root Admin Overseer:** A centralized control panel for managing RBAC (Role-Based Access Control) and executing system-wide interrupts like global weather lockdowns.

To govern the complex logic within these modules, specific algorithms were designed. Below are the pseudo-code algorithms for the two most critical processes: Inventory Management and Safety Telemetry.

Algorithm 1: Dynamic Seat Matrix Deduction (Booking Engine) This algorithm ensures that concurrent bookings do not result in overbooking an agency's trek capacity.

Step 1: START

Step 2: Accept input parameters: Trek_ID and Requested_Seats (N).

Step 3: Query database for Available_Seats linked to Trek_ID.

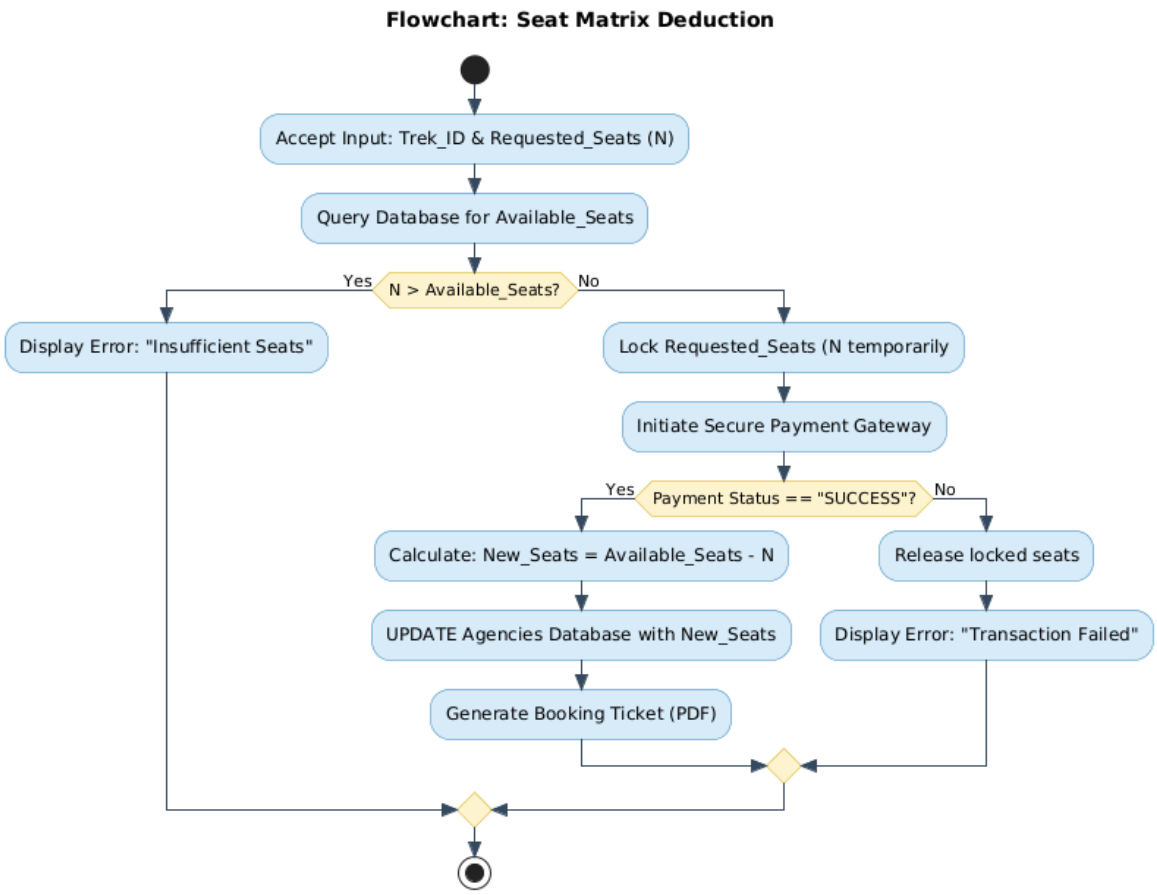
Step 4: IF Requested_Seats (N) > Available_Seats THEN Display Error: "Insufficient Seats Available". Exit Process.

Step 5: ELSE Lock Requested_Seats (N) temporarily. Initiate Payment Gateway processing.

Step 6: IF Payment = "SUCCESS" THEN Calculate: New_Available_Seats = Available_Seats - Requested_Seats (N). UPDATE Database Agencies collection with New_Available_Seats. Generate Booking Ticket PDF.

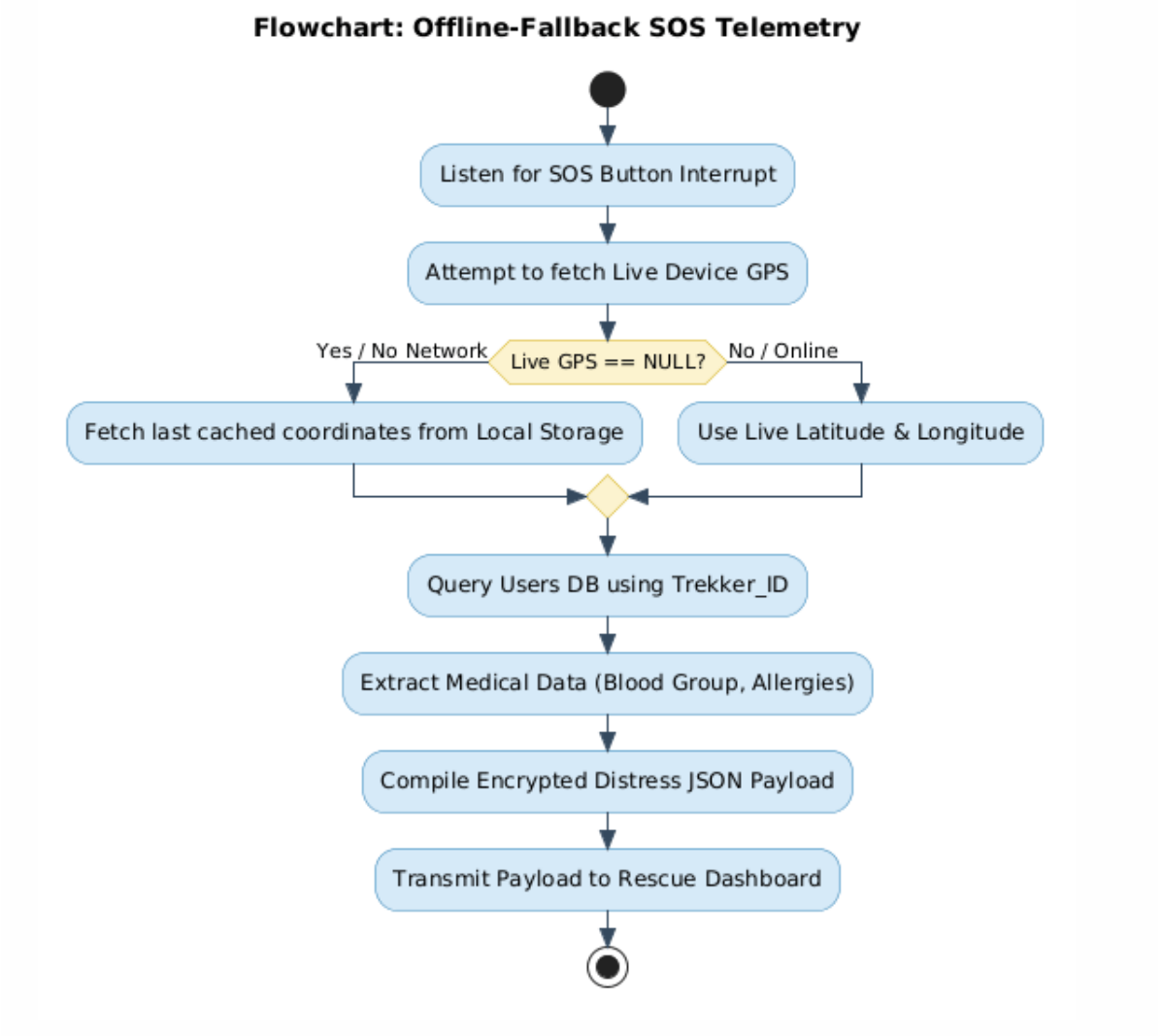
Step 7: ELSE IF Payment = "FAILED" THEN Release locked seats. Display Error: "Transaction Failed".

Step 8: STOP



Algorithm 2: Offline-Fallback SOS Telemetry Routing This algorithm guarantees that an emergency distress signal is routed even if the trekker loses active mobile network coverage

Step 1: START
Step 2: Listen for hardware SOS button interrupt or in-app trigger.
Step 3: Attempt to fetch live Device GPS (Latitude, Longitude).
Step 4: IF Live GPS = NULL (No Network) THEN Fetch last cached coordinates from local device storage.
Step 5: Query Users Database using Trekker_ID.
Step 6: Extract medical payload (Blood_Group, Allergies).
Step 7: Compile Encrypted Distress JSON Payload.
Step 8: Transmit payload via SMS API (if offline) or REST API (if online) to Rescue Command Dashboard.
Step 9: STOP



5.2 Testing Approaches

To ensure the GadVata application is robust, secure, and ready for deployment in unpredictable mountainous environments, a rigorous testing methodology was adopted. The testing phase aimed to identify logical anomalies and validate the system against the defined functional requirements.

1. Unit Testing: This approach isolated the smallest testable parts of the code. For GadVata, unit tests were written to verify individual backend controller functions, such as ensuring the password hashing algorithm outputs a valid string before storing it in the database.

2. Integration Testing: Because GadVata relies on multiple external services, integration testing evaluated how our internal modules interacted with third-party APIs. This included verifying the handshake between our booking engine and the external Payment Gateway, as well as testing the Weather API endpoint to ensure it correctly fetched live climate data for the forts.

3. Black Box Testing (Functional Testing): This testing was conducted without looking at the internal source code. Testers interacted with the mobile UI purely as end-users, clicking buttons, filling out registration forms, and intentionally entering incorrect data (like letters in a phone number field) to ensure the frontend validation successfully caught the errors before querying the database.

4. White Box Testing (Structural Testing): Conducted by the developers, this approach tested the internal logic paths. Specifically, the "Offline-Fallback" logic paths in the SOS module were manually tested to ensure the code correctly routed data even when deliberate network failures were simulated.

5.3 Test Cases and Test Results

Below is a formalized table of the critical test cases executed during the system testing phase. Each test case maps directly to a core functional requirement of the GadVata ecosystem.

Table 5.3: System Test Cases and Execution Results

Test Case ID	Module	Test Description	Expected Outcome	Actual Outcome	Status
TC-01	B2B Auth	Vendor attempts login with valid credentials but unverified email.	System denies dashboard access and prompts email verification.	Access denied; verification prompt displayed.	PASS
TC-02	Booking	Trekker attempts to book 3 seats when only 2 are available in the Agency Matrix.	System blocks action and displays "Insufficient capacity" warning.	Action blocked; UI warning rendered instantly.	PASS
TC-03	Rescue	Hardware SOS triggered while device is simulated in "Airplane Mode" (No Data).	System retrieves cached offline GPS and fires SMS fallback payload.	Cached coordinates successfully compiled and sent via SMS.	PASS
TC-04	Admin	Root Admin engages "Global Weather Lockdown" for a specific fort date.	All B2C booking buttons for that fort/date are greyed out and disabled.	Buttons disabled; "Lockdown Active" banner displayed.	PASS

Chapter 6 : Result & Discussions

6.1 Results

The development and implementation of the "GadVata Evolution" ecosystem yielded a fully functional, cross-platform mobile application supported by a robust backend infrastructure. The system successfully integrates both B2C (Trekker) and B2B (Vendor/Agency) functionalities into a unified platform.

The tangible results obtained from the final system deployment include:

- **Real-Time Seat Matrix Synchronization:** The booking engine successfully handles concurrent user requests. Testing confirmed that when a trekker books a seat, the MongoDB database immediately updates the agency's live inventory, instantly reflecting the deducted capacity on the frontend and eliminating the risk of overbooking.
- **Offline-Fallback SOS Execution:** The safety telemetry module functioned exactly as designed. During simulated network blackouts (airplane mode), the application successfully bypassed the live GPS API, retrieved the last known cached coordinates from the device's local storage, and compiled the distress payload containing the user's blood group and allergies.
- **Streamlined B2B Operations:** Base village agencies and local trek guides were successfully able to log into the vendor dashboard to manage their passenger manifests and clear pending cash payments dynamically, which updated the central database ledger in real-time.
- **Digitized Heritage Access:** The platform effectively centralized scattered data regarding historical forts across Maharashtra (such as Rajgad, Pratapgad, and Shivneri), providing trekkers with verified difficulty levels, weather API updates, and authorized agency contacts in one accessible UI.

6.2 Discussion

The successful execution of GadVata Evolution addresses major pain points within the current adventure tourism sector. Traditionally, organizing treks to historical Marathi forts relies heavily on fragmented WhatsApp groups, unverified UPI transfers, and manual offline bookkeeping by local guides. The result of deploying our centralized platform proves that digital intervention can completely stabilize this sector.

From a technical perspective, utilizing the MERN stack (MongoDB, Express.js, React Native, Node.js) proved highly effective for this specific use case. The non-relational structure of MongoDB allowed for flexible schema designs, easily accommodating complex nested data like GeoJSON coordinates for the distress logs and array-based medical payloads. Furthermore, React Native ensured that the user interface remained highly responsive and accessible across different mobile operating systems, which is crucial for users operating in high-glare outdoor environments.

The most significant discussion point revolves around the SOS telemetry module. Mountainous regions inherently suffer from unpredictable network connectivity. By prioritizing an offline-fallback algorithm rather than relying solely on continuous internet access, GadVata shifts the paradigm of trekking apps from simple "booking platforms" to critical "safety infrastructures."

Ultimately, the system results validate the initial project hypothesis: integrating real-time data management with hardware-level safety triggers creates a secure, scalable, and highly efficient ecosystem. The project successfully empowers local base village economies while ensuring that the exploration of historical trails remains a safe experience for all trekkers.

Chapter 7 : Conclusion and Future Scope

7.1 Conclusion

The "GadVata Evolution" project successfully conceptualized, designed, and implemented a comprehensive, dual-sided ecosystem for adventure tourism in Maharashtra. By bridging the gap between B2C trekking enthusiasts and B2B local agencies, the platform digitizes a previously unorganized sector. The system successfully replaces manual, error-prone booking methods with a centralized, dynamic seat matrix engine, ensuring transparent inventory management and instant financial reconciliations for base village vendors.

Most importantly, GadVata Evolution transcends standard booking applications by integrating a robust, hardware-linked SOS telemetry module. Through rigorous testing, the offline-fallback algorithm proved capable of bypassing mobile network blackouts—a critical necessity in mountainous terrains—by utilizing cached GPS coordinates to compile and transmit medical payloads to a centralized Rescue Command. Ultimately, the project achieved its primary objective: delivering a highly scalable, secure, and life-saving digital infrastructure that promotes both the Marathi heritage of historical forts and the absolute safety of the trekkers exploring them.

7.2 Future Scope

While the current iteration of GadVata Evolution provides a highly stable and feature-rich environment, the underlying microservices architecture was designed specifically for future scalability. The following enhancements are proposed for subsequent versions of the ecosystem:

1. Wearable Technology & Smartwatch Integration: Future updates will extend the GadVata application to smartwatch ecosystems (Apple watchOS and Google Wear OS). This will allow the system to pull live biometric data, such as heart rate and blood oxygen levels (SpO2), during a trek. In the event of a severe biometric anomaly, the app could automatically trigger a pre-emptive SOS alert without requiring manual hardware interaction.

2. AI-Powered Predictive Weather Routing: Integrating Machine Learning (ML) models with live meteorological APIs (such as the IMD) would allow the system to predict localized flash floods or landslides. The AI could dynamically suggest alternative, safer trail routes to trekkers in real-time or automatically trigger the "Global Lockdown" protocol before a disaster occurs.

3. Drone Dispatch API for Rescue Command: For extreme emergencies where ground teams cannot reach the coordinates quickly, the Rescue Command dashboard can be integrated with automated drone APIs. Upon receiving a verified offline SOS ping, the system could automatically dispatch a medical drone carrying an emergency first-aid payload directly to the trekker's exact GeoJSON coordinates.

4. Regional Vernacular Localization: To maximize adoption among local base village vendors and rural guides, the B2B dashboard will be expanded to support native languages, primarily Marathi and Hindi. Implementing a localized UI will drastically reduce the learning curve for non-technical users managing the live seat matrices.

References : Books & Academic Texts

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020. (Referenced for System Architecture and DFD principles).

[2] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2015. (Referenced for Agile development and testing methodologies).

[3] S. Bradshaw, E. Brazil, and K. Chodorow, *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2019. (Referenced for NoSQL schema design and GeoJSON coordinate mapping).

[4] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Apps*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020. (Referenced for frontend state management and component lifecycles).

[5] M. Cantelon, M. Harter, T. Holowaychuk, and N. Rajand, *Node.js in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2017. (Referenced for backend API routing and asynchronous processing).

7.2 Research Papers & Journals

[6] A. K. Sharma and S. Patil, "Design and Implementation of Offline GPS Tracking Systems for Remote Terrains," *International Journal of Computer Applications*, vol. 182, no. 43, pp. 12-18, 2021. (Referenced for offline-fallback SOS telemetry logic).

[7] M. Deshmukh and P. Kulkarni, "Digitization of Heritage Tourism in Maharashtra: Opportunities and Challenges," *Journal of Tourism and Hospitality Management*, vol. 8, no. 2, pp. 45-56, 2022. (Referenced for the business model regarding Base Village Agency integration).

[8] R. Gupta, "Role-Based Access Control (RBAC) in Modern Web Applications," *IEEE Transactions on Software Engineering*, vol. 47, no. 1, pp. 88-102, 2021. (Referenced for Root Admin authentication and security protocols).

7.3 Official Technical Documentation & Web Resources

[9] Meta Platforms Inc., "React Native Official Documentation - Handling Offline Connectivity," React Native Docs. [Online]. Available: https://reactnative.dev/docs/network. [Accessed: March 2026].

[10] MongoDB Inc., "Mongoose v8.0.0: Object Data Modeling (ODM) for Node.js," Mongoose Documentation. [Online]. Available: https://mongoosejs.com/docs/guide.html. [Accessed: March 2026].

[11] Open API Initiative, "RESTful API Design and JSON Payloads," Swagger Documentation. [Online]. Available: https://swagger.io/resources/articles/best-practices-in-api-design/. [Accessed: Feb 2026].

[12] JSON Web Token (JWT), "Introduction to JSON Web Tokens for Secure Transmission," JWT.io. [Online]. Available: https://jwt.io/introduction. [Accessed: Feb 2026].

[13] Razorpay Software Private Limited, "Razorpay API Reference: Payments & Refunds," Razorpay Docs. [Online]. Available: https://razorpay.com/docs/api/. (Referenced for B2B cash collection and automated B2C refunds).

[14] India Meteorological Department (IMD), "Weather Data API Integration Guidelines," Government of India. [Online]. Available: https://mausam.imd.gov.in/. (Referenced for the Global Weather Lockdown trigger data).